



TRABAJO FIN DE GRADO

Travily: red social de itinerarios de viaje

Murcia, a 29 de mayo de 2025

Juan Daniel López Hernández
48844364T
juandaniel.lopezh@um.es

Francisco García Sánchez

ÍNDICE DE CONTENIDOS

I.	Resumen.....	vi
II.	Extended abstract	vii
1	Introducción	1
2	Estado del arte	2
2.1	Análisis de la situación de partida	2
2.2	Tecnologías empleadas.....	5
2.2.1	<i>Back-end</i>	5
2.2.2	<i>Front-end</i>	6
2.2.3	Miscelánea.....	7
3	Análisis de objetivos y metodología.....	8
3.1	Objetivos.....	8
3.2	Metodología.....	8
4	Diseño y resolución del trabajo realizado	11
4.1	Análisis	11
4.1.1	Casos de uso para un usuario no registrado	11
4.1.2	Casos de uso para un usuario registrado	12
4.2	Diseño	14
4.2.1	Modelo de datos	15
4.2.2	Diseño del <i>back-end</i>	16
4.2.3	Diseño del <i>front-end</i>	18
4.3	Implementación.....	20
4.3.1	Implementación del <i>back-end</i>	20
4.3.2	Implementación del <i>front-end</i>	34
4.4	Pruebas	50
4.5	Despliegue	50
5	Conclusiones y vías futuras	52
5.1	Conclusiones	52
5.2	Vías futuras	52
6	Bibliografía	54

ÍNDICE DE FIGURAS

Figura 1. Aplicación web de Triplt	3
Figura 2. Aplicación web de Wanderlog.....	4
Figura 3. Aplicación web de Stippl	4
Figura 4. Diagrama de casos de uso para un usuario no registrado	11
Figura 5. Diagramas de casos de uso para (a) usuario registrado general, (b) propietario de itinerarios, (c) colaborador de itinerarios y (d) propietario de listas personalizadas	12
Figura 6. Arquitectura de Travily.....	14
Figura 7. Modelo de dominio de Travily	15
Figura 8. Diagrama de la base de datos	16
Figura 9. Mockup de la página principal (versión escritorio)	18
Figura 10. Mockup de la página principal (versión móvil)	19
Figura 11. Mockup de la página de un itinerario (versión escritorio).....	19
Figura 12. Diagrama de navegación de Travily	20
Figura 13. Estructura de carpetas del <i>back-end</i> de Travily	21
Figura 14. Archivo de configuración <code>.env</code>	21
Figura 15. Archivo de configuración <code>config.js</code>	22
Figura 16. Archivo principal de la aplicación (<code>app.js</code>).....	22
Figura 17. Conexión al pool en <code>database/db.js</code>	23
Figura 18. Método <code>create</code> del modelo <code>ItineraryModel</code>	24
Figura 19. Esquema de validación para la creación de un itinerario	25
Figura 20. Rutas del controlador de itinerarios	26
Figura 21. Método <code>create</code> del controlador de itinerarios	27
Figura 22. Ejemplo de token JWT y su decodificación	27
Figura 23. Método <code>login</code> del controlador de autenticación	28
Figura 24. Método <code>refreshToken</code> del controlador de autenticación	29
Figura 25. <i>Middleware</i> de autenticación.....	29
Figura 26. Configuración del servidor de WebSockets con <i>Socket.IO</i>	30
Figura 27. Unión a una sala y persistencia de cambios pendientes.....	31
Figura 28. Propagación en tiempo real de la edición de un evento	31
Figura 29. Persistencia periódica de cambios	32
Figura 30. Desconexión de un usuario y abandono de una sala	32
Figura 31. Configuración del almacenamiento en Cloudinary	33
Figura 32. Ruta de subida de un avatar de usuario.....	33
Figura 33. Estructura de carpetas del <i>front-end</i> de Travily.....	34
Figura 34. Archivo <code>main.tsx</code>	35
Figura 35. Componente <code>App.tsx</code>	36
Figura 36. Componente <code>ProtectedRoute</code>	37
Figura 37. Página de inicio de sesión	37
Figura 38. Formulario de inicio de sesión	38
Figura 39. Declaración de los estados para el formulario de inicio de sesión	38
Figura 40. Función <code>handleSubmit</code>	39
Figura 41. Método <code>login</code> en <code>AuthProvider</code>	39
Figura 42. Método <code>login</code> en <code>authService</code>	39

Figura 43. Página principal (versión escritorio).....	40
Figura 44. Página principal (versión móvil)	40
Figura 45. Funciones <code>fetchPopularItineraries</code> y <code>fetchFriendsItineraries</code>	41
Figura 46. Componente <code>ItinerariesCarousel</code>	42
Figura 47. Componente <code>ItineraryCard</code>	43
Figura 48. Página de detalles de un itinerario en modo edición.....	44
Figura 49. Modal para añadir un evento.....	45
Figura 50. Código para la carga inicial de datos del itinerario	46
Figura 51. Método <code>handleAddEvent</code>	47
Figura 52. Página de perfil de un usuario.....	47
Figura 53. Modal de seguidores de un usuario	48
Figura 54. Página de búsqueda	49
Figura 55. Aplicación Postman	50

ÍNDICE DE TABLAS

Tabla 1. Resumen comparativo de las soluciones analizadas	5
Tabla 2. Plan de trabajo: planificación mensual de las tareas	9
Tabla 3. Plan de trabajo: estimación de horas por tarea	10
Tabla 4. <i>Endpoints</i> de la aplicación	17

I. Resumen

La planificación de viajes se ha convertido en una actividad cada vez más digital, impulsada por herramientas online que permiten organizar, coordinar y compartir itinerarios. Sin embargo, muchas de estas soluciones no ofrecen una experiencia centralizada, colaborativa ni adaptada a las necesidades sociales del usuario. Ante esta necesidad, se ha desarrollado Travily, una aplicación web con enfoque de red social que permite a los usuarios crear, editar y compartir itinerarios de viaje de forma colaborativa y en tiempo real. Esta plataforma no solo busca facilitar la planificación de viajes, sino también fomentar la interacción entre usuarios y la creación de una comunidad de viajeros.

A partir del análisis de otras aplicaciones existentes, como Triplt, Wanderlog y Stippl, se ha identificado una serie de limitaciones que Travily pretende superar. Triplt permite organizar reservas de forma centralizada, pero no ofrece edición manual de actividades ni un enfoque de red social. Wanderlog incorpora edición colaborativa y un enfoque social, pero no distingue tipos de actividades ni permite ciertos filtros personalizados. Stippl ofrece herramientas organizativas, pero carece de una cronología de actividades y no permite personalizar completamente los itinerarios. Travily propone una solución que integra las mejores características de estas plataformas, junto con funciones como la creación de listas personalizadas, filtrado de itinerarios por usuarios seguidos, colaboración en tiempo real y un diseño minimalista e intuitivo que prioriza la experiencia de usuario.

La aplicación sigue una arquitectura cliente-servidor, compuesta por un *front-end* y un *back-end* que se comunican a través de una API RESTful. El *back-end* se ha desarrollado en JavaScript utilizando Node.js y Express, mientras que la persistencia de datos se gestiona mediante MySQL, con la librería mysql2. Para la validación de datos se emplea la librería Zod, y la colaboración en tiempo real se implementa con Socket.IO. Por su parte, el *front-end* está construido con React y TypeScript, utilizando Tailwind CSS y la librería Mantine para garantizar una interfaz adaptativa, moderna y coherente en distintos dispositivos.

El desarrollo se ha llevado a cabo siguiendo una metodología estructurada en cinco fases: análisis preliminar, análisis y diseño del sistema, desarrollo del sistema, verificación y validación, y documentación. La aplicación distingue entre usuarios no registrados y usuarios registrados, entre los cuales se incluyen propietarios y colaboradores de itinerarios, así como propietarios de listas. Cada rol dispone de funcionalidades específicas dentro de la plataforma.

La arquitectura del sistema se basa en el patrón MVC (Modelo-Vista-Controlador). Los modelos se encargan de la lógica de negocio y el acceso a base de datos, la vista corresponde a la interfaz de usuario, y los controladores actúan como intermediarios gestionando las peticiones HTTP y la validación. El sistema de autenticación usa JSON Web Tokens (JWT) almacenados en cookies *HttpOnly*, y las contraseñas se gestionan de forma segura mediante la librería bcrypt.

Travily también permite la subida de imágenes a través de Cloudinary, y ha sido desplegada usando Vercel para el *front-end*, Render para el *back-end*, y Aiven para la base de datos. Además, durante el desarrollo se realizaron pruebas continuas para verificar el correcto funcionamiento del sistema. Como posibles mejoras futuras se plantea la integración con APIs externas como Google Maps, chat privado para colaboradores de itinerarios, un sistema de notificaciones, comentarios en itinerarios y eventos, y soporte multilingüe.

II. Extended abstract

Travel planning is becoming increasingly digital, often involving the use of online tools and services for organising, coordinating, and sharing trips. Despite the abundance of available resources, users often face a fragmented planning experience. Most solutions either focus on booking management or itinerary visualisation but fail to provide a collaborative and socially connected environment. In response to this situation, Travily has been developed: a responsive web application with a social network approach that allows users to create, share and edit travel itineraries collaboratively in real time, making the entire planning process easier and more efficient from both an individual and group perspective. This platform not only improves the way users plan their trips, but also fosters a community around travel by encouraging user engagement and shared experiences.

The rise of platforms such as Triplt, Wanderlog and Stippl has demonstrated the demand for digital travel planning tools. Triplt focuses on gathering reservation information but lacks support for manually adding custom activities and does not provide a fully customisable planning experience. As for Wanderlog, it embraces a social network approach and allows real-time editing. However, it lacks some options for custom filtering and clear visual distinction between different types of activities. Regarding Stippl, it includes useful organisational features but does not offer a structured timeline of activities or support for creating personalized itineraries. Travily aims to overcome these limitations by combining the best features of existing tools while introducing enhancements such as creating custom itinerary lists, filtering itineraries created by followed users, real-time collaboration using WebSockets, and an intuitive, minimalistic design that prioritises user experience. This fusion of usability, flexibility and collaborative features places Travily in a distinct position within the field of itinerary planning applications.

To achieve the proposed idea, a client-server architecture was developed, made up of two main components: a front-end and a back-end, which communicate through HTTP requests. Each component was built using a specific set of programming languages, frameworks and libraries suited to its purpose, ensuring seamless integration and optimal performance.

On one hand, the back-end was implemented using JavaScript as the programming language, along with Node.js as the runtime environment and Express as the web framework, which simplifies the management of routes and middleware. This combination offers a flexible and modular foundation for building scalable and efficient RESTful APIs. MySQL is used as a reliable and high-performance relational database, accessed via the mysql2 library. In addition, real-time collaboration is supported by the Socket.IO library, which enables live, bidirectional communication using the WebSocket protocol.

On the other hand, the front-end was developed using React as the framework and TypeScript as the programming language. React enables the creation of reusable components and promotes a modular structure, while TypeScript adds static typing to enhance code reliability and reduce runtime errors. HTML is used to define the structure of the interface, and both Tailwind CSS and the Mantine UI library were included to provide a clean, responsive, and consistent design. This combination ensures a smooth and intuitive user experience across different devices and screen sizes.

The methodology applied in the project is structured into five main phases: (i) preliminary analysis, which involves researching existing solutions and the required technologies for front-end development (React, TypeScript, Tailwind CSS, Mantine UI) and back-end development (Node.js, Express, RESTful services), among other relevant tools and libraries; (ii) system analysis and design, including defining scope, establishing functional and non-functional requirements, and designing the database, the back-end using REST services, and responsive front-end components; (iii) system development, involving building the database schema, implementing back-end business logic, developing the front-end user interface, and integrating both parts; (iv) verification and validation, focusing on testing the system against specified requirements, populating the database with test data, and deploying the system; (v) system documentation, which involves documenting software components, preparing the thesis, and preparing the final presentation for the thesis defence.

After defining the methodology, the system scope and requirements were established through natural language use cases. These use cases are organized into subsections based on the actions available to each user role. The application distinguishes between unregistered and registered users. Unregistered users can browse public itineraries, view profiles, and access general information, while registered users enjoy additional features such as creating and managing itineraries, building personalized lists, interacting with other users, and customizing their profiles. Among registered users, itinerary owners have full control over their itineraries, while collaborators can modify content but are restricted from deleting itineraries, changing visibility or adding new collaborators. Owners of personalized lists have exclusive control over managing their lists.

The design of the application follows the MVC (Model-View-Controller) architecture pattern, which divides the system into three main components. The model is responsible for handling the business logic and database operations, as well as defining the entities of the system. The view corresponds to the user interface through which the user interacts. The controller acts as the link between the model and the view, receiving HTTP requests from the view, invoking operations on the model, and returning the appropriate JSON responses.

The domain model of the application centres around the `User` entity, who can create `Itineraries` that are associated with specific `Locations`. Each `Itinerary` can be divided into multiple `Days`, and each `Day` may include several `Events`. An `Itinerary` belongs to a single `User`, but it can be shared and edited collaboratively by other `Users` who are granted access as collaborators. In addition, `Users` can organise their `Itineraries` into personalized `Lists` and express their preferences by liking `Itineraries` and `Lists`. Social interaction is also supported through the option to follow other `Users`, allowing them to stay updated on their activity within the platform.

To enable communication between the front-end and back-end, a RESTful API has been designed with clearly defined endpoints. Each endpoint corresponds to one of the system entities and uses standard HTTP methods to read, create, update, and delete resources. This fully decoupled architecture allows the system to support multiple user interfaces and third-party clients across different platforms.

Additionally, a visual design phase was conducted using Figma to create mockups for both desktop and mobile versions of the platform. These mockups defined the structure of key pages (such as the home page, itinerary view, and user profile). A navigation diagram was also created to map navigation flows between pages, ensuring a smooth experience. This early design process helped establish a consistent and user-friendly interface adaptable to different screen sizes.

Following the design phase, we proceed with the system implementation. In terms of the back-end, the project is structured modularly into the following main directories: `controllers`, `models`, `schemas`, `middlewares` and `routes`. Other essential components of the system include authentication, file upload management, real-time communication using WebSockets and custom error handling. Each entity in the system has an associated controller and model, with no need for additional service or repository layers, ensuring separation of concerns and contributing to the system's maintainability and scalability.

Data persistence is managed through MySQL, connected via a connection pool established using the `mysql2` library. The models are responsible for executing SQL operations and handling transactions when necessary to ensure data integrity. The `Zod` library is used for schema-based data validation before interacting with the database, ensuring that all input complies with the expected structure and constraints.

The control flow in the back-end starts when an HTTP request is received from the client. The appropriate controller extracts the necessary parameters (from the URL, request body, or query), validates them against predefined schemas, and checks user authorization when applicable. Once validated, it calls the corresponding model method to execute the required database operations. After these operations complete, the controller builds and sends an HTTP response with the appropriate status code and JSON data. Error handling is centralized: exceptions or unexpected results from models are caught by the controllers and forwarded to an error-handling middleware, which responds with clear error messages and suitable HTTP status codes such as 400, 404, or 500.

The authentication system is implemented using JWTs (JSON Web Tokens), stored as `HttpOnly` cookies, providing secure, stateless user sessions. Users authenticate through a login endpoint which issues both an access token and a refresh token. When the access token expires, the refresh token allows the client to request a new one, enhancing both security and usability without compromising user experience. A middleware is used to validate tokens and restrict access to protected routes, ensuring that users can only perform actions allowed by their authentication status. Additionally, user passwords are securely hashed using the `bcrypt` library before being stored in the database.

Real-time collaborative editing is enabled by the `Socket.IO` library, which allows multiple users to join a virtual "room" corresponding to the itinerary they are editing. Updates (such as event creation or deletion) are instantly propagated to all connected users. Certain actions, like event reordering, are temporarily stored and saved in batches to optimise performance.

For image upload functionality, the system integrates with the Cloudinary cloud storage service to ensure reliability and scalability. Uploaded images (avatars, itinerary, event or list

images) are automatically resized to optimize performance and stored in organized folders for easy access. File handling is managed using the `multer` and `multer-storage-cloudinary` libraries.

The front-end of Travily is structured as a SPA (Single Page Application) using React. Each part of the interface is divided into reusable components located in structured folders such as `components`, `pages`, and `hooks`, ensuring maintainability and scalability. TypeScript enhances reliability by introducing static typing and reducing runtime errors. Tailwind CSS and Mantine provide a responsive and visually consistent design across different devices and screen sizes. Navigation is handled through React Router, and state management is implemented using local component state and context where appropriate. Communication with the back-end is managed through services that use the Fetch API to encapsulate HTTP requests, while WebSocket connections enable live updates between collaborators editing the same itinerary, ensuring that changes are immediately visible to all users. Whenever a user modifies a shared itinerary, changes are pushed to the server and broadcasted to other users in the same editing session.

The interface is organized around layout components that structure the application's pages consistently, such as the `MainLayout` component, which wraps the main routes and provides persistent interface elements including the header, footer, a bottom navigation bar for mobile devices, and side columns in the desktop version. These layouts also define the main content area, where more specific UI elements are rendered. The `Home` page, for example, is composed of multiple carousel sections, each displaying a set of `ItineraryCard` components.

During the development, the system was regularly tested to ensure that each component worked correctly in different scenarios. For the back-end, endpoints were tested using Postman to verify that each route returned the expected results, including handling invalid inputs and authentication errors. For the front-end, manual testing was performed on both desktop and mobile views, simulating user interactions and verifying interface responsiveness. The browser's developer tools were used to emulate different screen sizes and inspect real-time WebSocket communication.

As the final stage of the project, deployment was executed using Vercel for the front-end, enabling automatic builds from the GitHub repository. Meanwhile, the back-end was hosted on Render, which handles Node.js setup and secure environment variable management. For data storage, a managed MySQL database on Aiven was combined with Cloudinary for organized media file storage.

In conclusion, the proposed objectives have been successfully achieved through the development of Travily, a responsive web application for travel planning. The platform enables users to create and share itineraries in an intuitive and visually appealing environment, with a strong emphasis on user interaction and real-time collaboration, allowing multiple users to contribute simultaneously to the planning experience. While the current version focuses on core functionality, it lays a solid foundation for future improvements. Planned enhancements include the integration with third-party APIs like Google Maps, private chat for itinerary collaborators, a notification system, user comments on itineraries and events, and support for multiple languages. With further development, Travily could become a more solid and competitive solution within the travel planning field.

1 Introducción

La planificación de viajes es una actividad que, si bien puede ser emocionante, suele resultar abrumadora debido a la cantidad de información que se debe organizar. Muchas personas recurren a diversas fuentes y herramientas para coordinar sus itinerarios, buscar destinos de interés o compartir recomendaciones. A comienzos de 2024 se registró un aumento del 20% en la audiencia diaria de sitios web de viajes en España, lo que refleja la creciente digitalización en la planificación de viajes [1]. Sin embargo, en ocasiones la gestión de esta información resulta ineficiente, sobre todo cuando no existe una plataforma centralizada que permita a los viajeros colaborar y compartir sus planes de forma sencilla.

En este contexto, surge la necesidad de una plataforma que no solo facilite la creación y gestión de itinerarios de viaje, sino que también permita la colaboración en tiempo real entre usuarios, superando así las limitaciones de las herramientas tradicionales. Con este propósito, se ha desarrollado **Travily**, una aplicación web con enfoque de red social cuyo propósito es facilitar la interacción entre viajeros, ofreciendo un espacio donde puedan compartir sus experiencias y planificar juntos sus próximos destinos.

La plataforma está diseñada para ser intuitiva y accesible, permitiendo a los usuarios registrarse, personalizar su perfil, explorar itinerarios, crear y gestionar sus propios itinerarios de viaje, añadir días y eventos personalizados, editarlos en tiempo real junto a otros colaboradores y organizar itinerarios en listas personalizadas. Además, los usuarios pueden buscar destinos o perfiles, seguir a otros viajeros y marcar itinerarios o listas como favoritos. Estas funcionalidades consolidan a Travily como una solución integral para la planificación de viajes, destacando su enfoque social y colaborativo.

Es interesante señalar el origen del nombre de la aplicación, Travily, que es una combinación de las palabras en inglés "*travel*" (viajar) y "*ally*" (aliado).

La estructura del documento es la siguiente:

- **Capítulo 2 (Estado del arte):** se analiza el contexto previo al desarrollo de este proyecto, detallando las soluciones existentes en el mercado, así como sus principales fortalezas y debilidades. Además, se describen las tecnologías empleadas para el desarrollo de la plataforma.
- **Capítulo 3 (Análisis de objetivos y metodología):** se describen los objetivos que se persiguen con el desarrollo de este proyecto, así como la metodología empleada para alcanzar estos objetivos.
- **Capítulo 4 (Diseño y resolución del trabajo realizado):** se presenta el análisis de los casos de uso a implementar, el diseño y la arquitectura de la aplicación, la implementación detallada tanto del *back-end* como del *front-end*, las pruebas realizadas al sistema y el proceso de despliegue.
- **Capítulo 5 (Conclusiones y vías futuras):** se exponen las conclusiones del desarrollo de este proyecto, evaluando si se han alcanzado los objetivos propuestos. Además, se discuten posibles mejoras o funcionalidades futuras que podrían incrementar el valor de la plataforma.

2 Estado del arte

En este capítulo se analizará el contexto previo al desarrollo de Travily, evaluando las características y principales deficiencias de otras soluciones existentes en el ámbito de la planificación de viajes. Además, se describirán las tecnologías que se ha decidido emplear para la realización del proyecto.

2.1 Análisis de la situación de partida

Actualmente existen diversas aplicaciones web destinadas a facilitar la planificación de viajes, ofreciendo utilidades como compartir los itinerarios creados con otros usuarios.

Para este análisis, se han seleccionado tres aplicaciones representativas: Triplt, Wanderlog y Stipl. La elección de estas plataformas se ha basado en su visibilidad en los resultados de búsqueda de Google y su presencia destacada en artículos y blogs especializados, lo que refleja tanto su popularidad como su relevancia dentro del sector. Además, se ha tenido en cuenta su disponibilidad multiplataforma (iOS, Android y versión web) y la diversidad de enfoques que ofrecen respecto a la planificación de viajes, lo que permite obtener una visión amplia de las opciones disponibles para los usuarios.

En la evaluación, se han considerado las siguientes características clave:

- Funcionalidad de planificación de viajes: incluye la creación y edición de itinerarios.
- Enfoque de red social: capacidad para seguir a otros usuarios, consultar sus itinerarios y marcarlos como favoritos.
- Experiencia de usuario: facilidad de uso, diseño visual y organización de la información.
- Funcionalidades adicionales: como guías de viaje, buscador de hoteles, mapas integrados, gestión de gastos o herramientas colaborativas como chats o votaciones.

Una de las aplicaciones más conocidas es **Triplt** [2] (ver Figura 1), que destaca por su enfoque en las reservas de vuelos, hoteles, etc. Esta aplicación permite organizar viajes de manera rápida y sencilla al crear automáticamente un itinerario con todas las reservas asociadas a ese viaje, sin importar dónde se hayan realizado. La creación automática de este itinerario tiene lugar a partir del reenvío de los correos electrónicos de confirmación de las reservas a una dirección específica de la plataforma. La aplicación también permite compartir el itinerario con otros usuarios, ya sea con permisos de edición o solo de vista. Los usuarios con permiso de edición tienen también la posibilidad de reenviar sus correos de confirmación de reservas, añadiéndose al itinerario.

Esta aplicación logra su propósito de agrupar las reservas de un viaje en un mismo lugar, facilitando su organización, sin embargo, no presenta la posibilidad de añadir actividades manualmente a los itinerarios, tales como visitas a monumentos o comidas en restaurantes. Además, a pesar de que permite compartir itinerarios y su edición colaborativa, no adopta un enfoque de red social como propone Travily, cuyo propósito es incorporar funcionalidades como poder seguir a usuarios, consultar sus itinerarios, marcarlos como favoritos, entre otras.

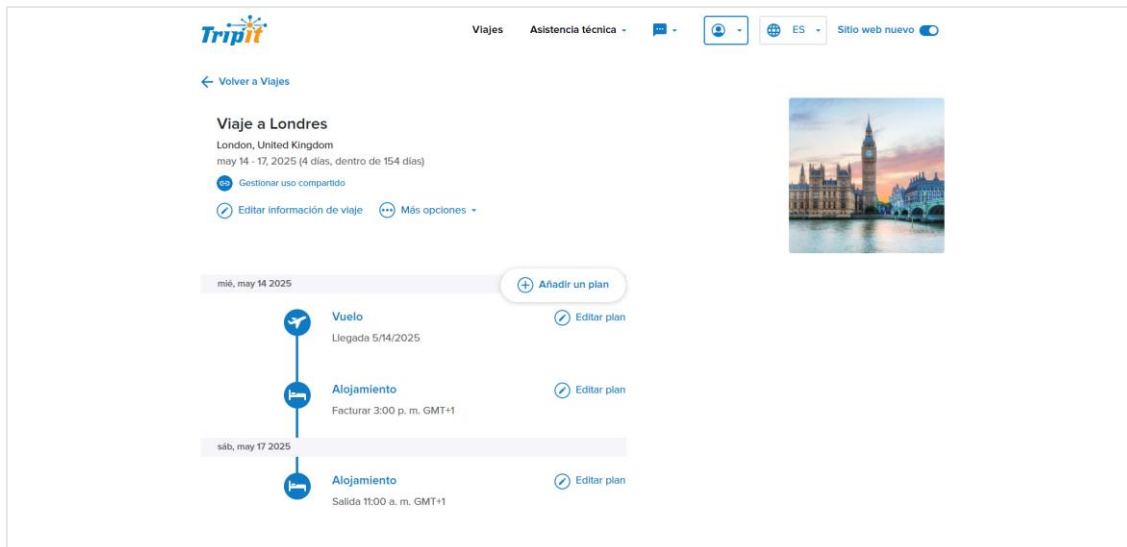


Figura 1. Aplicación web de Triplt

Otra alternativa muy popular es **Wanderlog** [3] (ver Figura 2), que presenta un enfoque de red social similar al de Travily. Esta plataforma permite crear itinerarios de viaje, compartirlos con otros usuarios y editarlos en tiempo real con más colaboradores. A diferencia de Triplt, Wanderlog sí permite agregar actividades manualmente al itinerario. Dispone también de guías de viaje con listas de monumentos, restaurantes y actividades recomendadas. Una característica distintiva de Wanderlog es la capacidad de poder ver en la misma pantalla el itinerario del viaje junto con un mapa donde se encuentran señalados los principales puntos de interés del viaje. Además, presenta otras funciones interesantes, como un buscador de hoteles o la posibilidad de llevar un seguimiento de los gastos realizados en el viaje, dividiendo el coste entre los participantes del viaje.

No obstante, en los itinerarios de Wanderlog resulta complicado distinguir visualmente las distintas categorías de actividades a realizar, como una comida en un restaurante o la visita a un monumento. Además, no permite filtrar los itinerarios de un destino para explorar únicamente los creados por los usuarios que sigues, lo que dificulta la búsqueda de planes acordes a tus intereses. Tampoco ofrece la opción de crear listas personalizadas de itinerarios, limitando así la organización de viajes según necesidades particulares. Travily surge con el objetivo de superar las deficiencias descritas implementando estas funcionalidades.

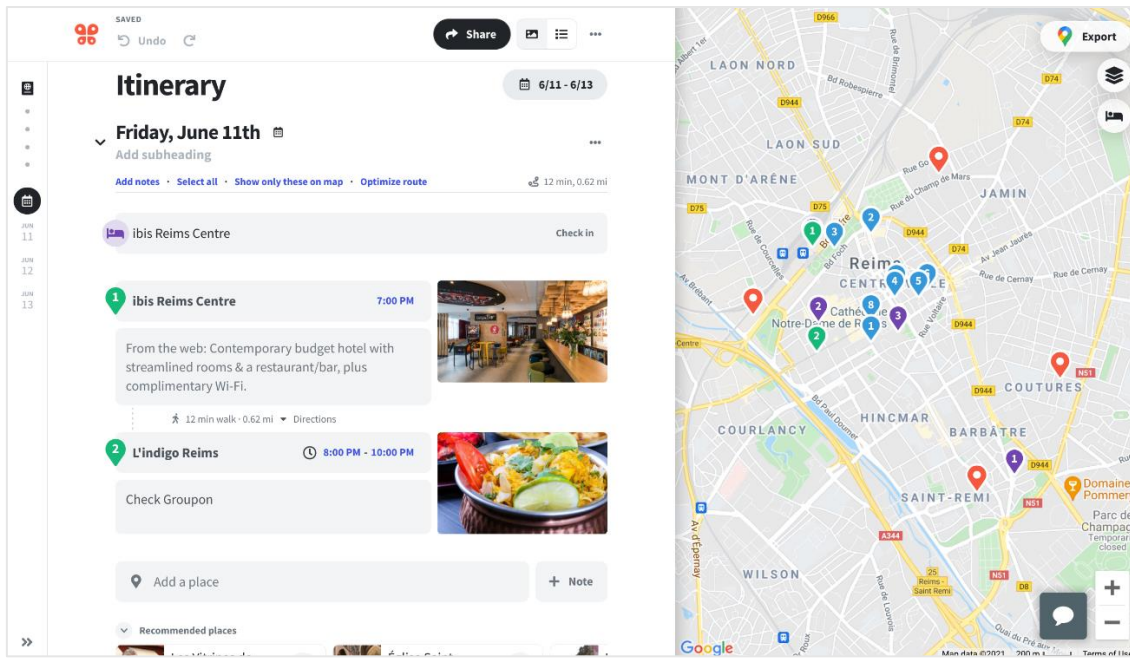


Figura 2. Aplicación web de Wanderlog

Stippl [4] (ver Figura 3) es otro ejemplo de aplicación diseñada para la planificación de viajes. Sin embargo, está más centrada en organizar las estancias y los transportes entre destinos del viaje que en ofrecer una cronología de las actividades a realizar. No es posible indicar el horario concreto en el que se van a llevar a cabo, tan solo el día. Por otro lado, la aplicación ofrece muchas otras funcionalidades, como un organizador de ropa para el viaje y un registro de gastos. Además, presenta algunas características de red social, como la opción de seguir a otros usuarios y consultar sus viajes y recomendaciones. No obstante, no cuenta con la funcionalidad de marcar viajes como favoritos, que Travily sí pretende implementar.

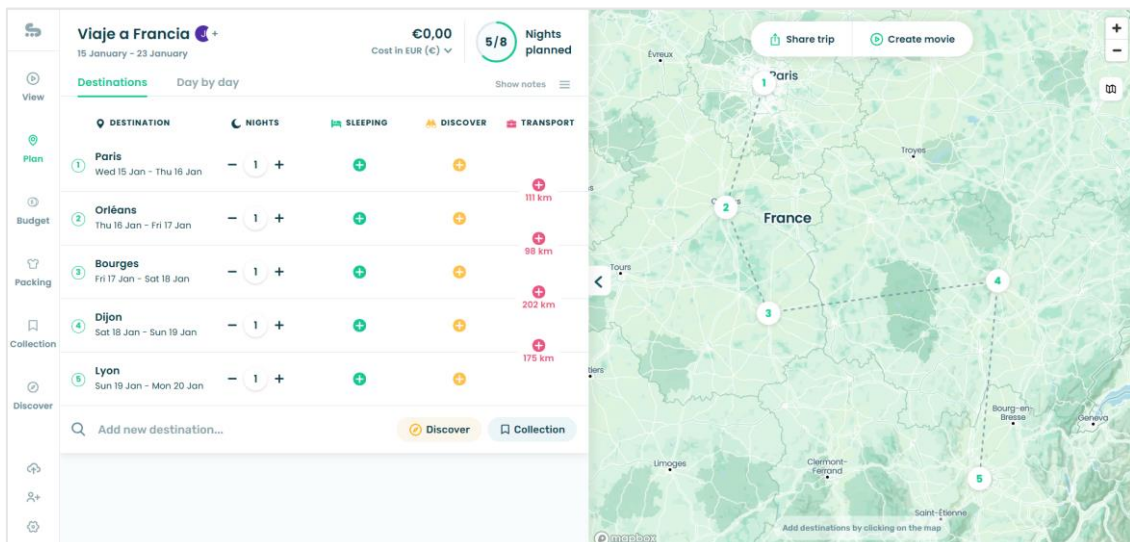


Figura 3. Aplicación web de Stippl

La aplicación desarrollada en este TFG, Travily, pretende incorporar las principales funcionalidades presentes en otras alternativas existentes, además de intentar solucionar algunas de las carencias encontradas. Propone una visión minimalista y simplificada de la

planificación de viajes, priorizando su enfoque como red social. Su diseño está orientado a ofrecer una experiencia sencilla e intuitiva, evitando que el usuario se sienta abrumado por un exceso de información y funcionalidades complejas. En la Tabla 1 se muestra un resumen comparativo de las soluciones analizadas.

Tabla 1. Resumen comparativo de las soluciones analizadas

Aplicación	Planificación de viajes	Red social	Experiencia de usuario	Funcionalidades adicionales
Triplt	Limitada (no permite añadir actividades manualmente)	No	Fácil de usar, diseño simple y funcional	Organización automática de reservas
Wanderlog	Amplia (permite añadir actividades y personalizarlas)	Sí	Atractiva pero algo compleja debido a la sobrecarga de funcionalidades e información y la dificultad de distinguir las categorías de las actividades	Guías de viaje, buscador de hoteles, mapas integrados, gestión de gastos
Stippl	Moderada (no ofrece una cronología detallada de las actividades)	Limitado (no permite marcar itinerarios como favoritos)	Fácil de usar, diseño atractivo y bien organizado	Gestión de gastos, organizador de ropa
Travily	Amplia (permite añadir actividades y personalizarlas)	Sí (prioriza este enfoque)	Fácil de usar, diseño minimalista e intuitivo, optimizado para interacción social	Filtrado de itinerarios por usuarios seguidos, listas personalizadas

2.2 Tecnologías empleadas

En este apartado se detallan los lenguajes de programación, *frameworks*, librerías y otras herramientas utilizadas en el desarrollo de la aplicación, abarcando tanto el lado del servidor (*back-end*) como el del cliente (*front-end*).

2.2.1 Back-end

Para el desarrollo del servidor, se ha elegido el lenguaje de programación **JavaScript** [5] debido a su flexibilidad y su capacidad de ejecutarse tanto en el lado del cliente como en el servidor. Este lenguaje multiparadigma, dinámico y de un solo hilo, permite utilizar diferentes estilos de programación, como la orientada a objetos, imperativa o declarativa. Al emplear un único lenguaje en toda la aplicación se facilita la comunicación entre ambos entornos y se mejora la coherencia del código. Además, JavaScript cuenta con un ecosistema amplio y una gran cantidad de librerías que agilizan el desarrollo.

Para la ejecución del código en el *back-end* se ha utilizado **Node.js** [6], un entorno de ejecución de JavaScript multiplataforma y de código abierto. Gracias a su arquitectura basada en eventos

y su modelo no bloqueante, Node.js permite manejar operaciones asíncronas de manera eficiente y gestionar múltiples conexiones simultáneamente, lo que lo hace especialmente adecuado para aplicaciones web dinámicas y escalables, así como para la creación de APIs RESTful [7].

Sobre esta base se ha utilizado **Express** [8], un *framework* minimalista y modular que facilita la gestión de rutas, peticiones HTTP y middleware. Proporciona las herramientas necesarias para crear APIs RESTful de forma eficiente y estructurada, simplificando el desarrollo.

Se ha empleado la librería **Socket.IO** [9] para implementar el protocolo **WebSocket** [10], permitiendo la comunicación en tiempo real entre los usuarios a través del servidor y, por tanto, la edición colaborativa de itinerarios en la aplicación.

Además, se ha incorporado la librería **Zod** [11] para las validaciones de datos mediante esquemas definidos, lo que asegura que la información recibida a través de las solicitudes HTTP cumpla con las estructuras y restricciones esperadas antes de ser procesada.

Para la gestión de dependencias, se ha empleado **npm** (del inglés “*Node Package Manager*”) [12], el sistema de administración de paquetes de Node.js. npm facilita la instalación y actualización de bibliotecas externas, manteniendo el proyecto modular y escalable.

En cuanto a la persistencia de datos, se ha optado por **MySQL** [13], el sistema de gestión de bases de datos relacional y de código abierto más popular del mundo, desarrollado por Oracle. Es ampliamente reconocido por su fiabilidad, rendimiento, escalabilidad y facilidad de uso. Dado que el proyecto implica gestionar relaciones complejas entre las entidades, el modelo relacional se adapta mejor que uno no relacional. Además, ofrece soporte para transacciones y mecanismos de integridad referencial, lo que asegura la consistencia de los datos [14].

2.2.2 Front-end

El *front-end* de la aplicación se ha implementado utilizando **React** [15] junto con **TypeScript** [16], una elección muy popular en el desarrollo web actual [17]. React, desarrollado por Meta, es una biblioteca de JavaScript que permite crear interfaces de usuario dinámicas mediante componentes reutilizables, lo que facilita la organización y modularidad del código, así como la escalabilidad del proyecto. Además, el uso del **Virtual DOM** [18] optimiza el rendimiento al actualizar solo las partes necesarias de la interfaz, evitando la recarga completa de la página y ofreciendo una experiencia de usuario más fluida. Por su parte, TypeScript es un superconjunto de JavaScript que extiende su sintaxis añadiendo tipado estático, mejorando la integración con herramientas de desarrollo y facilitando la detección temprana de errores, lo que contribuye a una mayor robustez y mantenibilidad del código.

Para la estructura del contenido web y el diseño de la interfaz, se ha empleado **HTML** (Lenguaje de Marcas de Hipertexto, del inglés “*HyperText Markup Language*”) [19] y **Tailwind CSS** [20], un *framework* de **CSS** (Hojas de Estilo en Cascada, del inglés “*Cascading Style Sheets*”) [21] basado en clases de utilidad predefinidas que permite aplicar estilos personalizados a cada elemento de forma rápida y sencilla, directamente en el marcado HTML. Este enfoque elimina la necesidad de archivos CSS extensos, agilizando el proceso de desarrollo.

Además, se ha integrado **Mantine** [22], una biblioteca de UI basada en Tailwind CSS que proporciona componentes de React prediseñados, mejorando la productividad y garantizando una consistencia visual en toda la aplicación. Estos componentes son *responsive*, adaptándose a distintas resoluciones de pantalla y asegurando una experiencia consistente en cualquier dispositivo.

2.2.3 Miscelánea

Además de las tecnologías mencionadas, se han empleado diversas herramientas que han facilitado el desarrollo y la gestión del proyecto:

- Para el desarrollo del código se ha elegido **Visual Studio Code** [23], un editor de código desarrollado por Microsoft, gratuito, de código abierto y compatible con múltiples lenguajes de programación. Su flexibilidad y soporte para extensiones le permiten funcionar como un entorno de desarrollo integrado (IDE, del inglés “*Integrated Development Environment*”) ligero, facilitando el trabajo con las distintas tecnologías utilizadas.
- En cuanto al diseño de la interfaz de usuario, se ha empleado **Figma** [24], una herramienta que permite la creación de *mockups* y prototipos interactivos de manera colaborativa.
- Para el diseño y gestión de la base de datos se ha utilizado **MySQL Workbench** [25], una herramienta que permite modelar esquemas relacionales, ejecutar consultas SQL y administrar bases de datos MySQL de manera visual e intuitiva.
- Las pruebas de la API REST desarrollada en el *back-end* se han llevado a cabo con **Postman** [26], una plataforma que facilita la validación de *endpoints*, permitiendo enviar solicitudes HTTP y verificar las respuestas del servidor de manera eficiente.
- El control de versiones se ha gestionado mediante **Git** [27], utilizando **GitHub** [28] como plataforma para alojar el repositorio del proyecto, lo que ha permitido mantener un historial detallado de los cambios en el código.
- Se ha utilizado **ChatGPT** [29] como asistente de inteligencia artificial generativa, ayudando a resolver dudas técnicas y optimizar el desarrollo.

3 Análisis de objetivos y metodología

En esta sección se describen los objetivos que se persiguen con el desarrollo de este proyecto, así como la metodología empleada (tareas y temporalización) para alcanzar estos objetivos.

3.1 Objetivos

El objetivo general que se plantea en este Trabajo Fin de Grado (TFG) es el desarrollo de una aplicación Web adaptable (*responsive*) para la planificación de viajes que permita crear y compartir itinerarios con otros usuarios. Este objetivo se puede dividir en los siguientes subobjetivos específicos:

- (OB1) Identificar las tecnologías y herramientas más apropiadas para acometer el trabajo
- (OB2) Determinar las principales propiedades y carencias de las soluciones actuales en el dominio de la aplicación
- (OB3) Desarrollar una aplicación que subsane las carencias identificadas satisfaciendo los principales estándares de calidad en el desarrollo de software
- (OB4) Desplegar la aplicación en un escenario de producción
- (OB5) Documentar el trabajo realizado

Es importante señalar que en este trabajo se persigue elaborar un primer prototipo de esta aplicación, dejando sin abordar aspectos como la integración con servicios externos y APIs de terceros de pago, como Google Maps, y la internacionalización para múltiples idiomas. Por otro lado, también se establece como objetivo de este TFG la realización de una memoria en que se describan las principales características del trabajo realizado.

3.2 Metodología

A continuación, se enumeran las etapas en las que se ha dividido la realización del TFG. Para cada tarea se establecen las subtarefas específicas y los objetivos abordados.

Tarea 1. Análisis preliminar

Objetivo(s): OB1, OB2

- 1.1. Consultar fuentes de información sobre creación de itinerarios de viaje
- 1.2. Estudiar el estado del arte sobre herramientas para la planificación de viajes
- 1.3. Estudiar las tecnologías para el desarrollo de un *front-end* adaptable (*responsivo*): React, TypeScript, Tailwind CSS, Mantine UI
- 1.4. Estudiar las tecnologías para el desarrollo del *back-end*: Node.js, Express, servicios REST
- 1.5. Estudiar herramientas y librerías que puedan ser útiles para el trabajo

Tarea 2. Análisis y diseño del sistema

Objetivo(s): OB2, OB3

- 2.1. Definir el alcance del sistema
- 2.2. Establecer los requisitos funcionales y no funcionales del sistema
- 2.3. Diseñar la base de datos del sistema
- 2.4. Diseñar el componente *back-end* del sistema a partir de servicios REST
- 2.5. Diseñar el componente *front-end* del sistema siguiendo un modelo *responsivo*

Tarea 3. Desarrollo del sistema

Objetivo(s): OB3

- 3.1. Crear el esquema de la base de datos
- 3.2. Desarrollar los servicios que constituyen la lógica de negocio (*back-end*)
- 3.3. Desarrollar los componentes que constituyen la interfaz de usuario adaptable (*responsive*) (*front-end*)
- 3.4. Conectar el *front-end* y el *back-end*

Tarea 4. Verificación y validación

Objetivo(s): OB3, OB4

- 4.1. Verificar el sistema frente a los requisitos especificados
- 4.2. Definir los casos de prueba
- 4.3. Poblar la base de datos con conjunto de datos necesarios para las pruebas
- 4.4. Validar el sistema a partir de los casos de prueba definidos
- 4.5. Desplegar el sistema en un entorno de producción

Tarea 5. Documentación del sistema

Objetivo(s): OB5

- 5.1. Reuniones periódicas con el tutor para revisar el estado del Trabajo Fin de Grado
- 5.2. Documentar internamente cada uno de los componentes software desarrollados
- 5.3. Elaborar la memoria del Trabajo Fin de Grado
- 5.4. Elaborar la presentación del Trabajo Fin de Grado

En la Tabla 2 se muestra la planificación temporal para la realización de las distintas tareas y subtareas, mientras que en la Tabla 3 se puede ver una estimación del tiempo que se invertirá en cada tarea.

Tabla 2. Plan de trabajo: planificación mensual de las tareas

Tarea	11/24	12/24	01/25	02/25	03/25	04/25	05/25	06/25
T1	X	X	X					
1.1	X							
1.2	X							
1.3	X	X						
1.4		X						
1.5		X	X					
T2		X	X					
2.1		X						
2.2		X						
2.3		X						
2.4		X	X					
2.5		X	X					
T3			X	X	X	X		
3.1			X					
3.2			X	X	X	X		
3.3			X	X	X	X		
3.4						X		
T4							X	
4.1							X	
4.2							X	
4.3							X	
4.4							X	
4.5							X	

T5	X	X	X	X	X	X	X	X
5.1	X	X	X	X	X	X	X	
5.2		X	X	X	X	X	X	
5.3	X	X	X	X	X	X	X	
5.4								X

Tabla 3. Plan de trabajo: estimación de horas por tarea

Tarea	Descripción	Horas estimadas
T1	Análisis preliminar	40
1.1	Consultar itinerarios de viaje	1
1.2	Estudiar estado del arte	4
1.3	Estudio tecnologías <i>front-end</i>	13
1.4	Estudio tecnologías <i>back-end</i>	15
1.5	Estudio herramientas y librerías	7
T2	Análisis/diseño sistema	40
2.1	Definir alcance del sistema	2
2.2	Establecer requisitos	5
2.3	Diseñar la BBDD	3
2.4	Diseñar <i>back-end</i>	10
2.5	Diseñar <i>front-end</i>	20
T3	Desarrollo de sistema	150
3.1	Crear esquema de la BBDD	5
3.2	Desarrollar el <i>back-end</i>	50
3.3	Desarrollar el <i>front-end</i>	80
3.4	Conectar <i>back-end</i> y <i>front-end</i>	15
T4	Verificación/Validación	30
4.1	Verificar los requisitos	2
4.2	Definir los casos de prueba	5
4.3	Poblar la base de datos	8
4.4	Validar el sistema	12
4.5	Desplegar el sistema	3
T5	Documentación	40
5.1	Reuniones con tutor	5
5.2	Documentar el desarrollo	8
5.3	Elaborar la memoria	20
5.4	Elaborar la presentación	7
Total		300

4 Diseño y resolución del trabajo realizado

En este capítulo se describe el proceso de análisis y diseño del prototipo de la aplicación, así como su desarrollo e implementación. También se incluyen las pruebas realizadas y el despliegue.

4.1 Análisis

En este apartado se analiza el alcance del sistema a través de la definición de sus requisitos, representados mediante casos de uso en lenguaje natural. La aplicación distingue entre dos categorías principales de usuarios: **usuarios no registrados** y **usuarios registrados**. Los usuarios no registrados pueden explorar itinerarios públicos, ver perfiles de usuarios y acceder a información general de la aplicación. En cambio, los usuarios registrados tienen acceso a una experiencia más completa, que incluye la creación y gestión de itinerarios, la creación de listas personalizadas, la interacción con otros usuarios y la personalización de su perfil.

Dentro de los usuarios registrados, se distinguen los **propietarios de itinerarios**, con permisos completos para gestionar itinerarios, y los **colaboradores de itinerarios**, quienes pueden modificar su contenido, pero no eliminarlos, cambiar su visibilidad ni agregar nuevos colaboradores. Además, los **propietarios de listas** personalizadas tienen acceso exclusivo para gestionar estas listas.

En los siguientes subapartados se detallan los casos de uso que reflejan las acciones disponibles para cada tipo de usuario en el sistema: usuarios no registrados (ver apartado 4.1.1) y usuarios registrados agrupados según su rol (propietarios de itinerarios, colaboradores de itinerarios y propietarios de listas) (ver apartado 4.1.2).

4.1.1 Casos de uso para un usuario no registrado

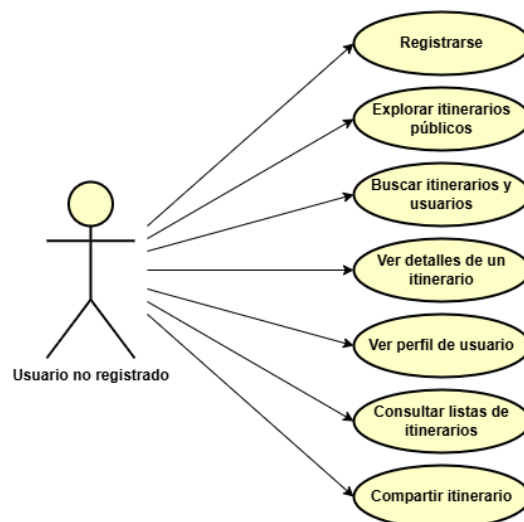


Figura 4. Diagrama de casos de uso para un usuario no registrado

En la Figura 4 se presentan gráficamente los casos de uso asociados a un usuario no registrado, enumerados a continuación:

- **CdU 1 - Registrarse:** El usuario puede registrarse en la aplicación proporcionando nombre, nombre de usuario, email, contraseña y, opcionalmente, foto de perfil.
- **CdU 2 - Explorar itinerarios públicos:** El usuario puede consultar todos los itinerarios públicos disponibles en la plataforma.
- **CdU 3 - Buscar itinerarios y usuarios:** El usuario puede buscar itinerarios filtrando por localización y usuarios en función de su nombre o nombre de usuario.
- **CdU 4 - Ver detalles de un itinerario:** El usuario puede acceder a la información detallada de un itinerario público.
- **CdU 5 - Ver perfil de usuario:** El usuario puede consultar su perfil y el de cualquier otro usuario.
- **CdU 6 - Consultar listas de itinerarios:** El usuario puede consultar la lista de itinerarios creados, los marcados como favoritos y las listas personalizadas, tanto propias como de otros usuarios, siempre que sean públicos.
- **CdU 7 - Compartir itinerario:** El usuario puede compartir un itinerario copiando la URL del itinerario al portapapeles.

4.1.2 Casos de uso para un usuario registrado

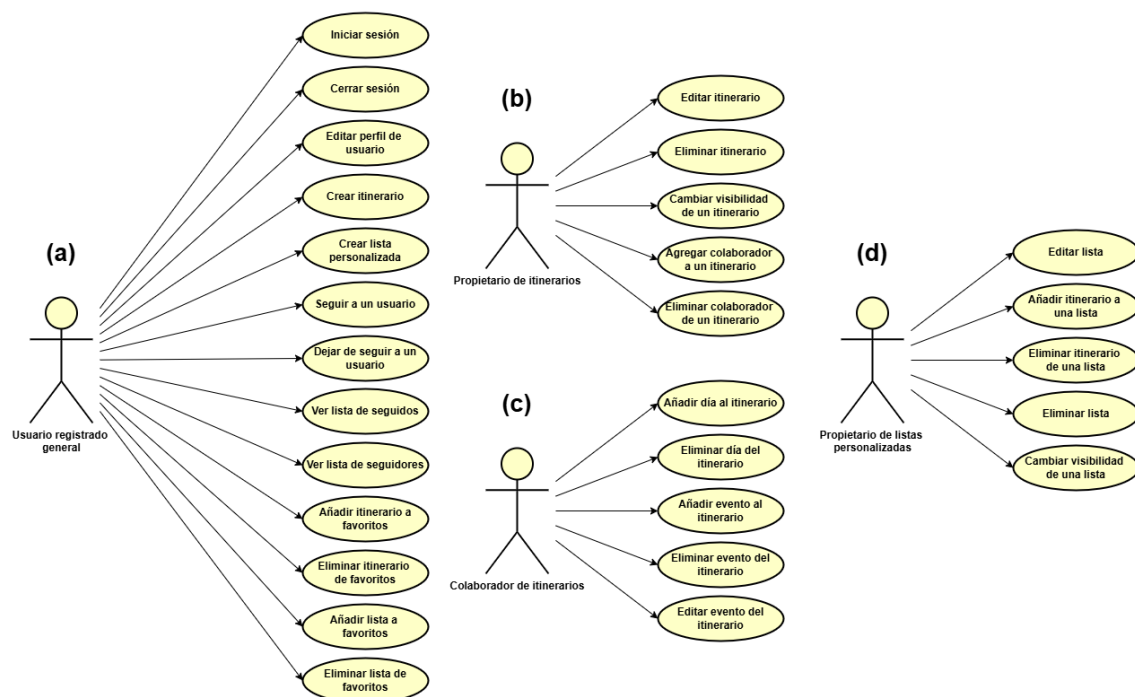


Figura 5. Diagramas de casos de uso para (a) usuario registrado general, (b) propietario de itinerarios, (c) colaborador de itinerarios y (d) propietario de listas personalizadas

En la Figura 5 se presentan gráficamente los casos de uso asociados a un usuario registrado, que se organizan en función del rol específico del usuario, como se indica a continuación.

4.1.2.1 Casos de uso generales

- **CdU 8 - Iniciar sesión:** El usuario registrado puede autenticarse en la aplicación introduciendo su nombre de usuario o email y contraseña.
- **CdU 9 - Cerrar sesión:** El usuario autenticado puede cerrar sesión en la aplicación.

- **CdU 10 - Editar perfil de usuario:** El usuario autenticado puede modificar la información de su perfil: nombre, nombre de usuario, email, contraseña y foto de perfil.
- **CdU 11 - Crear itinerario:** El usuario autenticado puede crear un nuevo itinerario proporcionando título, descripción (opcional), destino, fechas de inicio y fin, visibilidad (público o privado) e imagen (opcional).
- **CdU 12 - Crear lista personalizada:** El usuario autenticado puede crear una nueva lista de itinerarios personalizada, proporcionando título, descripción (opcional) e imagen (opcional).
- **CdU 13 - Seguir a un usuario:** El usuario autenticado puede seguir a otro usuario, añadiéndolo a la lista de seguidos en su perfil.
- **CdU 14 - Dejar de seguir a un usuario:** El usuario autenticado puede dejar de seguir a otro usuario, eliminándolo de la lista de seguidos en su perfil.
- **CdU 15 - Ver lista de seguidos:** El usuario autenticado puede ver la lista de usuarios a los que sigue, así como las listas de seguidos de otros usuarios.
- **CdU 16 - Ver lista de seguidores:** El usuario autenticado puede ver la lista de usuarios que lo siguen, así como las listas de seguidores de otros usuarios.
- **CdU 17 - Añadir itinerario a favoritos:** El usuario autenticado puede marcar un itinerario como favorito, añadiéndolo a la lista de favoritos en su perfil.
- **CdU 18 - Eliminar itinerario de favoritos:** El usuario autenticado puede eliminar un itinerario de su lista de favoritos.
- **CdU 19 - Añadir lista a favoritos:** El usuario autenticado puede marcar una lista de itinerarios personalizada como favorita, añadiéndola a la lista de favoritos en su perfil.
- **CdU 20 - Eliminar lista de favoritos:** El usuario autenticado puede eliminar una lista de itinerarios personalizada de su lista de favoritos.

4.1.2.2 Casos de uso para propietarios de itinerarios

- **CdU 21 - Editar itinerario:** El propietario de un itinerario puede modificar su información: título, descripción, fechas de inicio y fin e imagen.
- **CdU 22 - Eliminar itinerario:** El propietario de un itinerario puede eliminarlo.
- **CdU 23 - Cambiar visibilidad de un itinerario:** El propietario de un itinerario puede cambiar su visibilidad (público o privado).
- **CdU 24 - Agregar colaborador a un itinerario:** El propietario de un itinerario puede invitar a otros usuarios registrados como colaboradores.
- **CdU 25 - Eliminar colaborador de un itinerario:** El propietario de un itinerario puede retirar a un colaborador.

4.1.2.3 Casos de uso para colaboradores de itinerarios

- **CdU 26 - Añadir día al itinerario:** El propietario o colaborador de un itinerario puede añadir un nuevo día al itinerario.
- **CdU 27 - Eliminar día del itinerario:** El propietario o colaborador de un itinerario puede eliminar un día del itinerario.
- **CdU 28 - Añadir evento al itinerario:** El propietario o colaborador de un itinerario puede añadir un nuevo evento a un día del itinerario, proporcionando título,

descripción (opcional), horas de inicio y fin (opcional), categoría (opcional) e imagen (opcional).

- **CdU 29 - Eliminar evento del itinerario:** El propietario o colaborador de un itinerario puede eliminar un evento de un día del itinerario.
- **CdU 30 - Editar evento del itinerario:** El propietario o colaborador de un itinerario puede modificar los detalles de un evento en un día del itinerario: título, descripción, horas de inicio y fin, categoría e imagen.

4.1.2.4 Casos de uso para propietarios de listas

- **CdU 31 - Editar lista:** El propietario de una lista puede modificar su información: título, descripción e imagen.
- **CdU 32 - Añadir itinerario a una lista:** El propietario de una lista personalizada puede añadir un itinerario a la lista.
- **CdU 33 - Eliminar itinerario de una lista:** El propietario de una lista personalizada puede eliminar un itinerario de la lista.
- **CdU 34 - Eliminar lista:** El propietario de una lista puede eliminarla.
- **CdU 35 - Cambiar visibilidad de una lista:** El propietario de una lista puede cambiar su visibilidad (pública o privada).

4.2 Diseño

El diseño de la aplicación sigue una arquitectura cliente-servidor de tres capas: presentación, aplicación y persistencia. Esta arquitectura se muestra en la Figura 6. La capa de **presentación** gestiona la interfaz de usuario y su interacción con la aplicación. La capa de **aplicación** maneja la lógica de negocio y el procesamiento de los datos. Finalmente, la capa de **persistencia** se encarga del almacenamiento y la gestión de los datos en la base de datos.

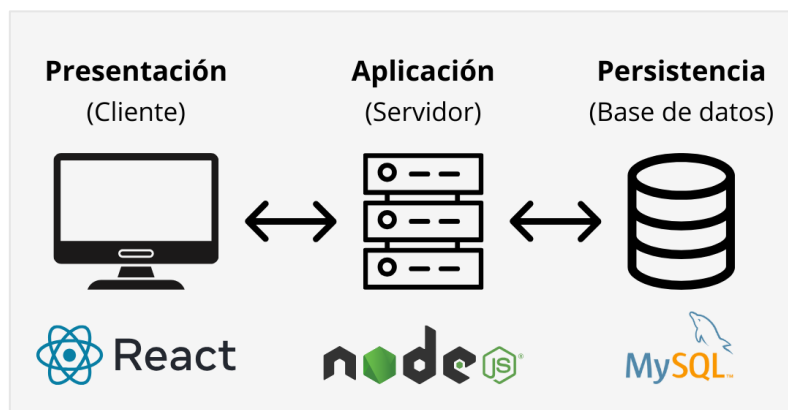


Figura 6. Arquitectura de Travily

Este enfoque se corresponde con el patrón de arquitectura **MVC** (Modelo-Vista-Controlador) [30], que promueve una separación clara de responsabilidades, facilitando la escalabilidad y el mantenimiento del sistema. La aplicación se organiza en tres componentes principales: modelo, vista y controlador.

El **modelo** es el encargado de gestionar la lógica de negocio y la manipulación de los datos de la aplicación. Define las entidades que forman la base de datos, sus relaciones y las reglas que garantizan su integridad. Además, implementa las operaciones CRUD (Crear, Leer, Actualizar,

Eliminar) necesarias para el funcionamiento del sistema. Es completamente independiente de la interfaz de usuario, centrándose únicamente en el procesamiento y almacenamiento de la información.

La **vista** representa la interfaz gráfica con la que interactúa el usuario. Su función es mostrar la información de manera estructurada a través de botones, formularios y otros elementos visuales. No incluye lógica de negocio, solo se encarga de presentar los datos y capturar las acciones del usuario.

El **controlador** actúa como intermediario entre la vista y el modelo. Recibe las solicitudes del usuario, las procesa y se comunica con el modelo para obtener o modificar los datos según sea necesario. Posteriormente, envía la respuesta adecuada a la vista para actualizar la interfaz. En el controlador se ha definido la API REST, estableciendo los *endpoints* que permiten la comunicación entre el *front-end* y el *back-end*.

A continuación, se detallarán los aspectos específicos del modelo de datos, así como el diseño del *back-end* y del *front-end*.

4.2.1 Modelo de datos

En la Figura 7 podemos observar un diagrama simplificado del modelo de dominio de la aplicación, en el que solo se muestran las entidades y sus relaciones, ya que los atributos se mostrarán más adelante en el diagrama de la base de datos.

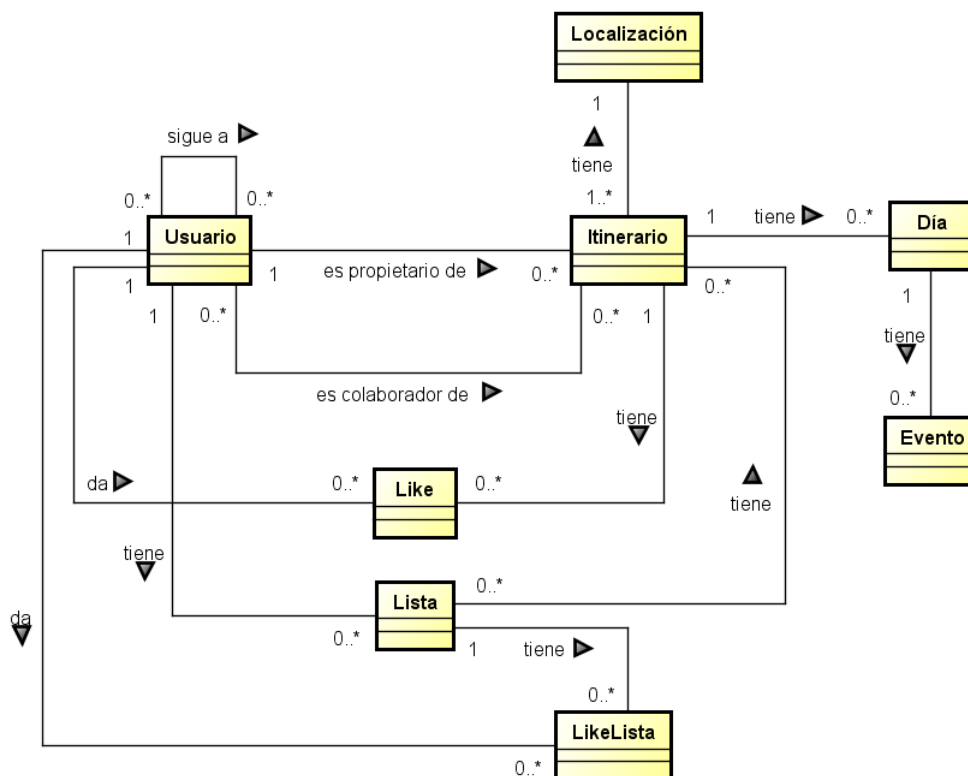


Figura 7. Modelo de dominio de Travily

La entidad principal del dominio de la aplicación es el **Usuario**, con la capacidad de crear **Itinerarios**, los cuales deben incluir una **Localización**. Los **Itinerarios** se pueden estructurar en

varios **Días**, y cada **Día** puede incluir múltiples **Eventos**. Un **Itinerario** es propiedad de un único **Usuario**, pero puede ser compartido y modificado por otros **Usuarios** que actúan como colaboradores. Además, los **Usuarios** pueden organizar sus **Itinerarios** en **Listas**, que pueden contener varios **Itinerarios**, y pueden dar "me gusta" tanto a los **Itinerarios** (**Like**) como a las **Listas** (**Likelista**). Asimismo, los **Usuarios** pueden interactuar entre sí mediante la funcionalidad de seguir a otros **Usuarios**, lo que les permite estar al tanto de sus novedades.

En la Figura 8 se muestra el esquema completo de la base de datos de Travily, generado automáticamente desde la herramienta MySQL Workbench [25]. Este esquema presenta en detalle las entidades, sus atributos y las relaciones establecidas entre ellas.

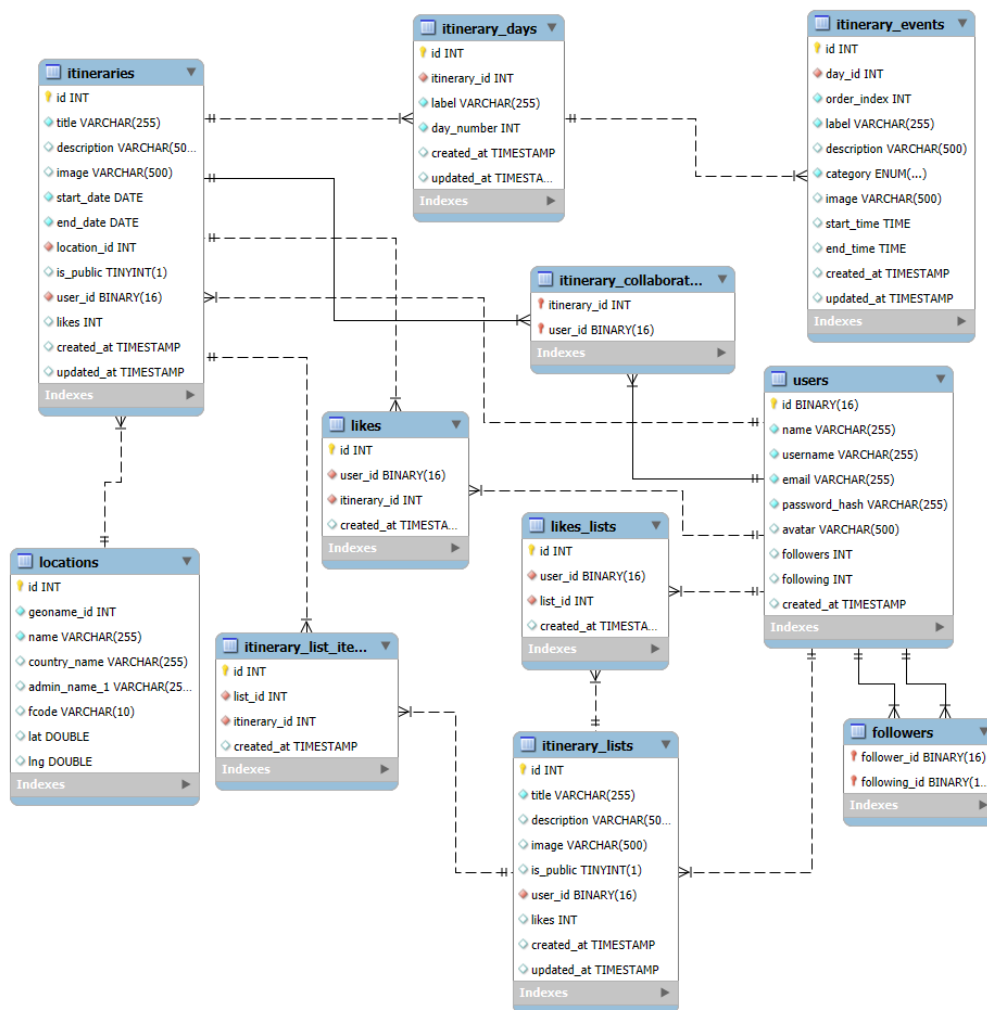


Figura 8. Diagrama de la base de datos

4.2.2 Diseño del *back-end*

El diseño del *back-end* de la aplicación sigue el patrón MVC, abarcando específicamente los componentes Controlador y Modelo. Este enfoque incluye la implementación de una API REST, que proporciona una interfaz para la comunicación con la vista. Los controladores actúan como intermediarios, procesando las peticiones de los usuarios y comunicándose con los modelos, que gestionan la lógica de negocio. Los modelos representan las entidades del sistema y realizan las operaciones CRUD necesarias en la base de datos.

Es importante destacar que el *back-end* de la aplicación está completamente desacoplado del *front-end*, lo que permite interactuar con el sistema a través de la API REST sin depender de una interfaz de usuario específica. Esto posibilita la creación de distintas interfaces de usuario adaptadas a diversas plataformas o necesidades, así como la interacción de clientes de terceros con el sistema.

La **API REST** gestiona las peticiones mediante solicitudes HTTP, donde cada recurso se identifica de forma única a través de su URL (*endpoint*). Los métodos HTTP permiten interactuar con estos recursos realizando diferentes acciones: **GET** para recuperar datos, **POST** para crear nuevos recursos, **PUT** para actualizar completamente recursos existentes, **PATCH** para realizar modificaciones parciales y **DELETE** para eliminar recursos. En la Tabla 4, se describen los *endpoints* disponibles y las operaciones que pueden realizar.

Tabla 4. *Endpoints* de la aplicación

Endpoint	GET	POST	DELETE	PATCH
/auth/register		✓		
/auth/login		✓		
/auth/logout		✓		
/auth/refresh-token		✓		
/users	✓	✓		
/users/suggested	✓			
/users/{identifier}	✓			
/users/{id}			✓	✓
/users/{id}/follow		✓	✓	
/users/{id}/is-following	✓			
/users/{id}/followers	✓			
/users/{id}/following	✓			
/itineraries	✓	✓		
/itineraries/{id}	✓		✓	✓
/itineraries/{id}/likes		✓	✓	
/itineraries/{id}/is-liked	✓			
/itineraries/{id}/collaborators	✓	✓		
/itineraries/{id}/is-collaborator	✓			
/lists	✓	✓		
/lists/{id}	✓		✓	✓
/lists/{id}/likes		✓	✓	
/lists/{id}/is-liked	✓			
/lists/{id}/itineraries		✓	✓	
/search/locations	✓			
/search/users	✓			
/upload/avatar		✓		
/upload/itinerary-image		✓		
/upload/event-image		✓		
/upload/list-image		✓		

4.2.3 Diseño del front-end

Para definir el diseño de la interfaz de usuario de Travily, se han realizado diversos *mockups* para dispositivos móviles y de escritorio, empleando la herramienta Figma [24].

En la Figura 9 podemos observar el *mockup* de la página principal de la aplicación en su versión de escritorio. En primer lugar, la pantalla presenta un *header* o barra superior con el logo de la aplicación a la izquierda y un icono de perfil a la derecha, que despliega un menú con las opciones “Mi perfil”, “Configuración” y “Cerrar sesión”. En cuanto al cuerpo de la página, se organiza en tres columnas: la columna izquierda ofrece accesos directos a “Mi perfil”, “Itinerarios”, “Favoritos”, “Listas” y “Crear itinerario”; la columna central muestra el contenido principal; y la columna derecha presenta sugerencias de usuarios a seguir. Las columnas izquierda y derecha permanecen fijas en todas las páginas de la aplicación.

En la columna central, se muestra un título de bienvenida junto a una barra de búsqueda que permite localizar itinerarios en destinos concretos o usuarios por nombre. A continuación, se disponen varios carruseles que organizan contenido relevante, como itinerarios populares de la semana, destinos destacados e itinerarios recientes de amigos. Los itinerarios se muestran mediante tarjetas que incluyen información útil como el título, descripción, imagen, destino, duración, perfil del propietario, número de “me gusta” recibidos y la opción de dar “me gusta”. Cada tarjeta es clicable y redirige a una página con todos los detalles del itinerario, mientras que el perfil del propietario también es clicable y lleva a su página de usuario. Al final de la página aparece un *footer* con diversos enlaces de interés.

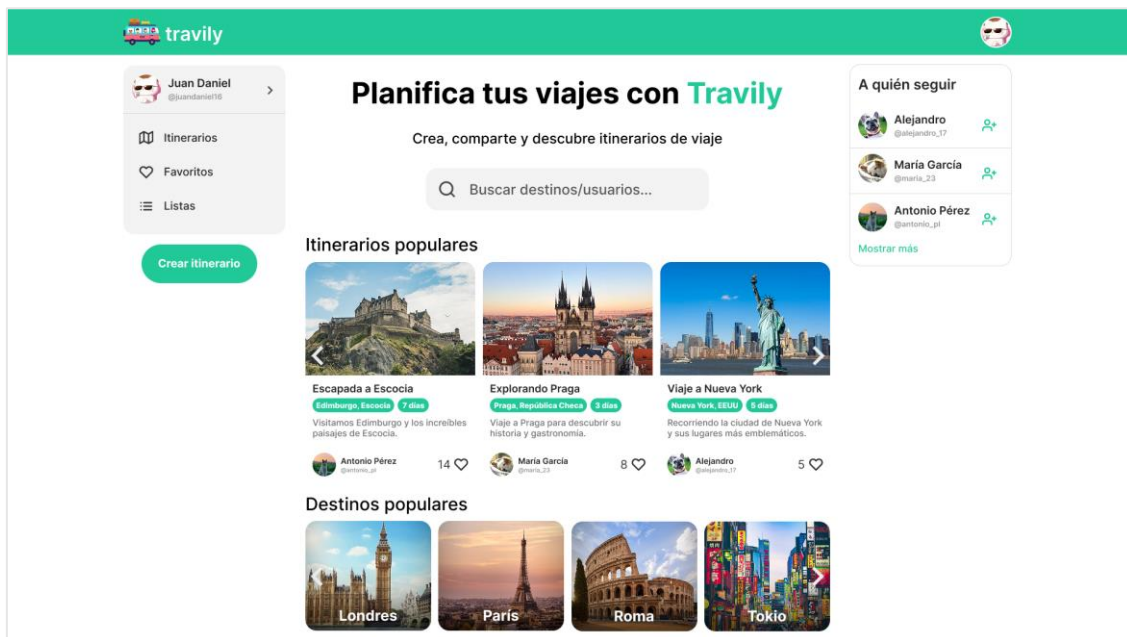


Figura 9. Mockup de la página principal (versión escritorio)

En la versión móvil, mostrada en la Figura 10, solo aparece la columna central, y en la parte inferior de la pantalla se encuentra una barra de navegación con accesos directos a 'Inicio', 'Explorar', 'Crear nuevo itinerario', 'Amigos' y 'Mi perfil'. El diseño responsive de la aplicación permite que se adapte a distintos tamaños de pantalla, asegurando una experiencia óptima en todos los dispositivos.

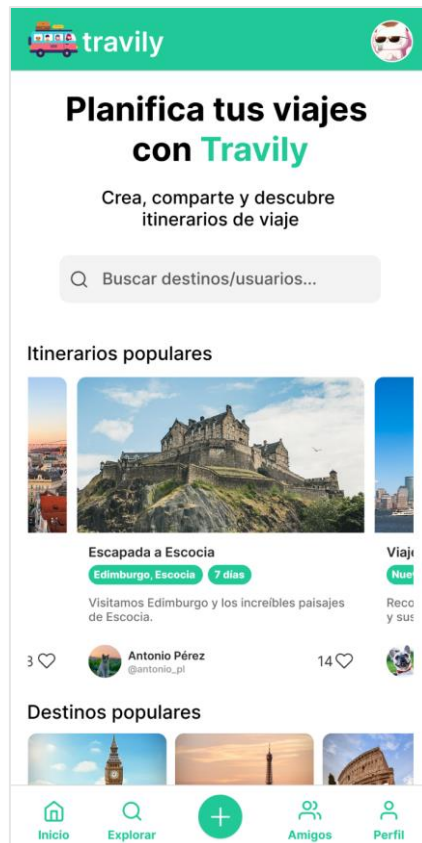


Figura 10. Mockup de la página principal (versión móvil)

La Figura 11 muestra el *mockup* de la página de un itinerario completo, donde se presenta toda su información, incluyendo sus días y los eventos programados para cada día. Además, se trata de la vista de edición del itinerario, que incluye botones para agregar y eliminar días o eventos, así como editar los eventos existentes.

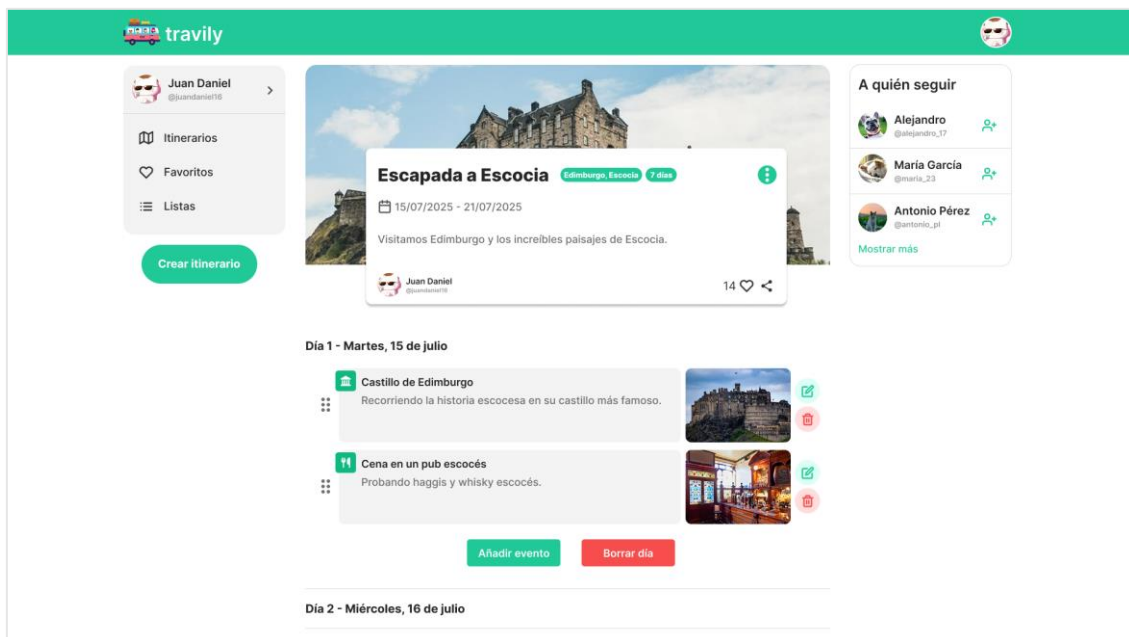


Figura 11. Mockup de la página de un itinerario (versión escritorio)

A continuación, en la Figura 12, se muestra un diagrama de navegación de Travily, en el que podemos observar cómo se interconectan las diversas pantallas o páginas dentro de la aplicación. El diagrama ilustra los flujos principales de navegación, destacando las acciones que permiten a los usuarios desplazarse entre las distintas secciones de la aplicación, como la página principal, el perfil de usuario, la página de un itinerario y otras funcionalidades clave. Además, muestra cómo los modales se enlazan con las pantallas principales, ofreciendo una visión clara de la estructura y el recorrido que los usuarios seguirán al interactuar con la interfaz.

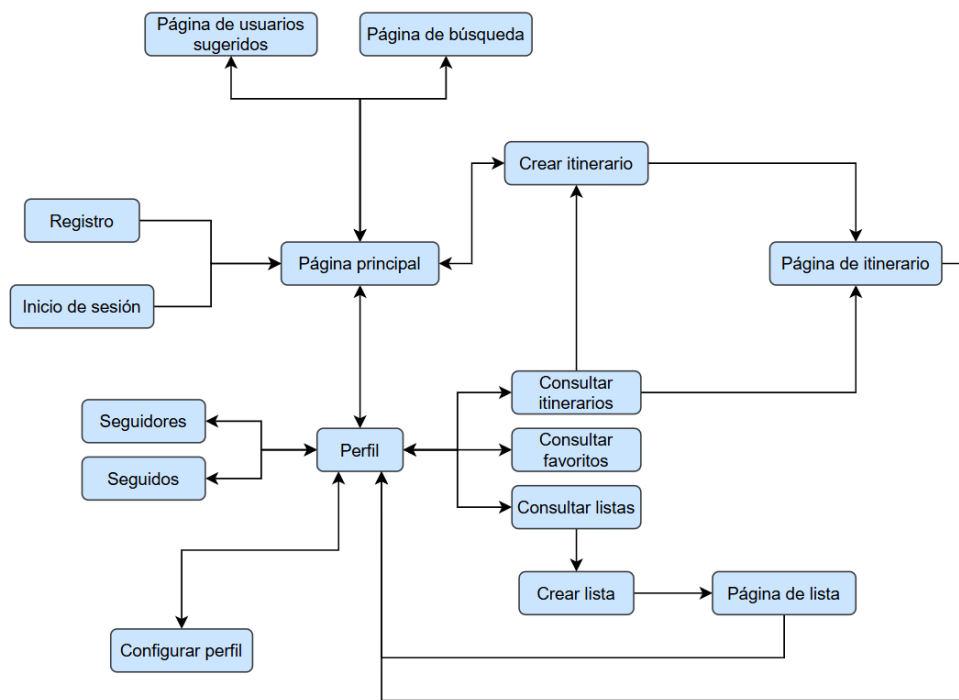


Figura 12. Diagrama de navegación de Travily

4.3 Implementación

En este apartado se detalla la implementación del *back-end* (servidor) y el *front-end* (cliente) de la aplicación Travily. El código fuente completo está disponible en el repositorio público de GitHub: <https://github.com/juandaniellh16/Travily>

4.3.1 Implementación del *back-end*

El *back-end* de la aplicación se ha desarrollado siguiendo una estructura modular que favorece la separación de responsabilidades, facilitando así tanto el mantenimiento como la escalabilidad del sistema. En la Figura 13 se muestra la organización de carpetas del proyecto, donde se distinguen los principales módulos que conforman el sistema: controladores, modelos, esquemas de validación, middlewares y rutas, así como otros elementos necesarios (seguridad de la aplicación, gestión de la carga de archivos, comunicación en tiempo real mediante WebSockets, manejo de errores personalizados, etc.).

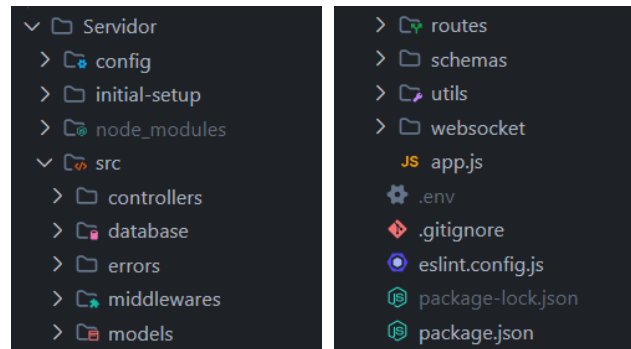


Figura 13. Estructura de carpetas del *back-end* de Travily

Cada una de las entidades principales del dominio cuenta con su propia clase controlador y clase modelo. En este proyecto, no se ha considerado necesario incorporar capas adicionales como servicios o repositorios, ya que la lógica del sistema se gestiona adecuadamente mediante una interacción directa entre controladores y modelos.

El resto de la estructura del servidor se organiza en módulos complementarios que desempeñan funciones específicas. En `schemas/` se definen los esquemas de validación implementados con la biblioteca **Zod** [10], garantizando la integridad de los datos recibidos. La carpeta `middlewares/` incluye funciones reutilizables como la verificación del token JWT, la validación de permisos o el manejo centralizado de errores. En `routes/` se agrupan los distintos archivos de rutas, uno por cada tipo de recurso, encargados de conectar cada *endpoint* con su controlador correspondiente. El módulo `websocket/` gestiona la comunicación en tiempo real entre usuarios mediante WebSockets, fundamental para la edición colaborativa de itinerarios. Por su parte, `utils/` contiene funciones auxiliares de utilidad general, mientras que `errors/` incluye clases de error personalizadas para un manejo más claro y estructurado de excepciones. Por último, la carpeta `uploads/` se emplea únicamente como directorio de almacenamiento para las imágenes subidas por los usuarios, sin contener lógica de aplicación.

La configuración general del *back-end* de la aplicación se gestiona mediante dos archivos principales: el archivo `.env` y el archivo `config.js`. El archivo `.env` especifica variables de entorno esenciales como la URL base de la API, el puerto del servidor, las credenciales de acceso a la base de datos y el secreto para los JWT. Estas variables se utilizan en todo el sistema, proporcionando flexibilidad y seguridad a la configuración. Por su parte, el archivo `config.js`, ubicado en la carpeta `config/`, utiliza la biblioteca **dotenv** [31] para cargar las variables definidas en el archivo `.env`, y se encarga de exportar las configuraciones necesarias. Los detalles de estos archivos se muestran en la Figura 14 y Figura 15.

```
API_BASE_URL=http://localhost:3000
PORT=3000
DATABASE_URL=
FRONTEND_URL=http://localhost:5173
JWT_SECRET=
SALT_ROUNDS=10
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

Figura 14. Archivo de configuración `.env`


```
import dotenv from 'dotenv'

dotenv.config()

export const PORT = process.env.PORT || 3000
export const API_BASE_URL = process.env.API_BASE_URL

const DEFAULT_CONFIG = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'root',
  database: 'itineraries_db'
}

export const FRONTEND_URL = process.env.FRONTEND_URL

export const DATABASE_URL = process.env.DATABASE_URL ?? DEFAULT_CONFIG
export const JWT_SECRET = process.env.JWT_SECRET
export const SALT_ROUNDS = process.env.SALT_ROUNDS || 10
```

Figura 15. Archivo de configuración `config.js`

Por otro lado, en la Figura 16 podemos observar el archivo `app.js`, situado en la raíz del directorio `src/`, que actúa como punto de entrada de la aplicación. En él se inicializa el servidor **Express** y se registran los middlewares necesarios, entre ellos los de configuración de CORS¹, autenticación, procesamiento de cookies y manejo de errores. Además, se definen las distintas rutas del sistema, incluyendo las de autenticación, usuarios, itinerarios, listas y búsqueda. También se inicializa WebSocket y se configuran las rutas para la carga de archivos y el acceso a los recursos estáticos.

```
const app = express()
const server = http.createServer(app)
initializeWebSocket({ server, dayModel: DayModel, eventModel: EventModel })
app.disable('x-powered-by')
app.use(corsMiddleware())
app.use(json())
app.use(cookieParser())
app.use(verifyOrigin)
app.use(auth)

app.use('/auth', createAuthRouter({ userModel: UserModel }))
app.use('/users', createUserRouter({ userModel: UserModel }))
app.use('/itineraries', createItinerariesRouter({ itineraryModel: ItineraryModel }))
app.use('/lists', createItineraryListsRouter({ itineraryListModel: ItineraryListModel }))
app.use('/search', createSearchRouter({ searchModel: SearchModel }))

app.use('/upload', uploadRouter)

app.use(errorHandler)

server.listen(PORT, () => {
  console.log(`Server listening on port http://localhost:\${PORT}`)
})
```

Figura 16. Archivo principal de la aplicación (`app.js`)

En los siguientes apartados se detallan los distintos componentes que forman parte de la implementación del *back-end*, incluyendo la persistencia de datos, los controladores, el sistema de autenticación de usuarios, la comunicación en tiempo real mediante WebSockets y la gestión de la carga de archivos.

¹ CORS (Cross-Origin Resource Sharing) es un mecanismo que permite controlar qué dominios pueden acceder a los recursos del servidor desde el navegador. Su configuración es necesaria en aplicaciones web donde el cliente y el servidor se encuentran en orígenes distintos.

4.3.1.1 Persistencia

Para la persistencia de los datos de la aplicación se ha utilizado MySQL [13], un sistema de gestión de bases de datos relacional que permite estructurar la información mediante tablas y relaciones entre ellas.

El *back-end* se conecta con esta base de datos a través de un pool de conexiones asíncrono proporcionado por la biblioteca **mysql2** [32]. Esta conexión se inicializa una única vez y se reutiliza durante toda la ejecución de la aplicación, optimizando el rendimiento y evitando la sobrecarga de apertura de múltiples conexiones. En la Figura 17 se muestra cómo se establece la conexión al pool en el archivo `database/db.js`.

```
const sslConfig = {
  ca: fs.readFileSync('./config/ca.pem')
}

const pool = mysql.createPool({
  uri: DATABASE_URL,
  ssl: sslConfig
})

export const getConnection = async () => {
  try {
    const connection = await pool.getConnection()
    return connection
  } catch (error) {
    console.error('Error connecting to the database: ', error)
    throw error
  }
}
```

Figura 17. Conexión al pool en `database/db.js`

A diferencia de otras arquitecturas que incorporan capas específicas para repositorios o utilizan ORMs [33], en este proyecto se ha optado por realizar la interacción con la base de datos directamente desde los modelos, que encapsulan la lógica de negocio y ejecutan las operaciones SQL necesarias para manipular las entidades en la base de datos.

A continuación, se divide el apartado de persistencia en dos bloques principales: el **Modelo**, que representa la lógica de negocio y las operaciones sobre la base de datos, y la **Validación de datos** antes de la persistencia, que asegura que solo los datos válidos sean enviados al modelo para su procesamiento.

Modelo

Las clases modelo de la aplicación están organizadas dentro de la carpeta `models/`. Cada modelo representa una entidad funcional del sistema, como usuarios, itinerarios, eventos, listas, etc. Los modelos no solo proporcionan los métodos CRUD estándar para realizar operaciones de lectura, escritura, actualización y eliminación, sino también otros métodos específicos que implementan la lógica de negocio necesaria para cada entidad.

Por ejemplo, el modelo `ItineraryModel` define un método `create`, mostrado en la Figura 18, que se encarga de insertar un nuevo itinerario en la base de datos. Para garantizar la consistencia de los datos, este método realiza varias operaciones secuenciales dentro de una transacción. Primero, inserta o actualiza la localización del itinerario a partir del objeto `location`, que contiene los datos geográficos previamente extraídos de **GeoNames** [34], una API pública que ofrece información detallada sobre lugares del mundo e identificadores únicos como `geonameId`. Este objeto se construye en el cliente al rellenar el formulario de creación

de itinerario, donde se solicita introducir un destino y se muestran sugerencias obtenidas a través de la API de GeoNames. Una vez seleccionado un destino, la información correspondiente se incluye en el cuerpo de la solicitud enviada al servidor.

A continuación, el método recupera el identificador de la localización y, finalmente, crea el itinerario asociado. Si alguna de estas operaciones falla, se revierte la transacción para evitar resultados parciales o inconsistencias, lanzando además una excepción personalizada mediante la clase `DatabaseError`, definida en el módulo `errors/`.

```
static async create({ input }) {
  const { title, description, image, startDate, endDate, location, isPublic, userId } = input

  const connection = await getConnection()
  try {
    await connection.beginTransaction()
    await connection.query(
      `INSERT INTO locations (geoname_id, name, country_name, admin_name_1, fcode, lat, lng)
      VALUES (?, ?, ?, ?, ?, ?, ?)
      ON DUPLICATE KEY UPDATE
        name = VALUES(name),
        country_name = VALUES(country_name),
        admin_name_1 = VALUES(admin_name_1),
        fcode = VALUES(fcode),
        lat = VALUES(lat),
        lng = VALUES(lng);`
      , [location.geonameId, location.name, location.countryName, location.adminName1, location.fcode, location.lat, location.lng]
    )

    const [locationIdResult] = await connection.query(
      `SELECT id FROM locations WHERE geoname_id = ?;`,
      [location.geonameId]
    )

    if (locationIdResult.length === 0) {
      throw new DatabaseError('Location not found after insert/update: ' + locationIdResult.message)
    }
    const locationId = locationIdResult[0].id

    const [itineraryResult] = await connection.query(
      `INSERT INTO itineraries (title, description, image, start_date, end_date, location_id, is_public, user_id)
      VALUES (?, ?, ?, ?, ?, ?, ?, UUID_TO_BIN(?));`
      , [title, description, image, startDate, endDate, locationId, isPublic, userId]
    )
    if (!itineraryResult.insertId) {
      throw new DatabaseError('Failed to create itinerary: ' + itineraryResult.message)
    }

    await connection.commit()
    return itineraryResult.insertId
  } catch (error) {
    await connection.rollback()
    throw new DatabaseError('Error creating itinerary: ' + error.message)
  } finally {
    connection.release()
  }
}
```

Figura 18. Método `create` del modelo `ItineraryModel`

Entre los modelos definidos en el sistema se encuentran también `UserModel`, `DayModel`, `EventModel`, `ItineraryListModel` y `SearchModel`, cada uno encargado de manejar las operaciones específicas de su dominio de datos.

Validación de datos

Antes de enviar los datos al modelo, se realiza una validación estricta desde los controladores. Para ello, se emplean esquemas definidos con la biblioteca **Zod** [11], ubicados en la carpeta `schemas/`. Estos esquemas permiten comprobar que los datos recibidos cumplen con la estructura y restricciones esperadas (campos obligatorios, tipos correctos, formatos válidos, etc.). En el caso de los itinerarios, por ejemplo, se utiliza un esquema que valida campos como el título, la imagen, la fecha de inicio, la localización o el identificador del propietario, como se muestra en la Figura 19.

```
const itinerarySchema = z.object({
  title: z.string({
    invalid_type_error: 'Itinerary title must be a string',
    required_error: 'Itinerary title is required'
  }),
  description: z.string({ invalid_type_error: 'Itinerary description must be a string' }).optional(),
  image: z.string({ invalid_type_error: 'Itinerary image must be a string' })
    .url('Itinerary image must be a valid URL')
    .nullable()
    .optional(),
  startDate: z.preprocess(
    (arg) => typeof arg === 'string' ? new Date(arg) : arg,
    z.date({
      invalid_type_error: 'Itinerary startDate must be a valid date',
      required_error: 'Itinerary startDate is required'
    })
  ),
  endDate: z.preprocess(
    (arg) => typeof arg === 'string' ? new Date(arg) : arg,
    z.date({
      invalid_type_error: 'Itinerary endDate must be a valid date',
      required_error: 'Itinerary endDate is required'
    })
  ),
  location: locationSchema,
  isPublic: z.boolean({ invalid_type_error: 'Itinerary isPublic must be a boolean' }).optional(),
  userId: z.string({
    invalid_type_error: 'Itinerary userId must be a string',
    required_error: 'Itinerary userId is required'
  })
})
```

Figura 19. Esquema de validación para la creación de un itinerario

Solo si los datos superan esta validación son enviados al modelo para su persistencia. En caso contrario, el sistema lanza una excepción personalizada del tipo `InvalidInputError`, evitando así que se almacenen datos inválidos o corruptos en la base de datos. Este enfoque asegura la integridad y coherencia de los datos almacenados, y al mismo tiempo mejora la experiencia del usuario al proporcionar respuestas claras en caso de errores.

4.3.1.2 Controlador

La capa de controladores actúa como intermediaria entre la vista (cliente) y la lógica de negocio implementada en los modelos. Es la encargada de recibir las solicitudes HTTP dirigidas a los *endpoints* de la API REST, extraer los parámetros necesarios, aplicar las validaciones correspondientes y delegar la lógica de negocio a los modelos, que realizan las operaciones SQL necesarias. Finalmente, construye y devuelve una respuesta adecuada al cliente.

En esta aplicación, los controladores se han definido como clases y se encuentran en la carpeta `controllers/`. Cada controlador está asociado a una entidad concreta del sistema (por ejemplo, itinerarios, usuarios, eventos, etc.), y se encarga de gestionar las operaciones relacionadas con dicha entidad, como crear, obtener, actualizar o eliminar datos. Cabe destacar que también se han definido controladores destinados a tareas específicas, como la autenticación de usuarios. Cada controlador tiene acceso al modelo correspondiente, el cual encapsula las operaciones sobre la base de datos. El controlador recibe una instancia del modelo a través de su constructor, lo cual favorece la reutilización del código y facilita las pruebas, ya que permite sustituir el modelo por uno simulado (*mock*).

Las rutas de la API REST están diseñadas siguiendo los principios RESTful [7], y se definen utilizando el enrutador de Express (`Router()`). Para cada entidad, se crea un archivo de rutas que expone los *endpoints* adecuados y los vincula con los métodos definidos en el controlador correspondiente. Por ejemplo, las rutas asociadas al controlador de itinerarios se muestran en la Figura 20. Este *router* se monta en el archivo `app.js` con el prefijo `/itineraries`, de modo que no es necesario especificarlo en las rutas internas.

```
export const createItinerariesRouter = ({ itineraryModel }) => {
  const itinerariesRouter = Router()

  const itineraryController = new ItineraryController({ itineraryModel })

  itinerariesRouter.get('/', itineraryController.getAll)
  itinerariesRouter.post('/', itineraryController.create)
  itinerariesRouter.get('/:id', itineraryController.getById)
  itinerariesRouter.delete('/:id', itineraryController.delete)
  itinerariesRouter.patch('/:id', itineraryController.update)

  itinerariesRouter.post('/:id/likes', itineraryController.like)
  itinerariesRouter.delete('/:id/likes', itineraryController.unlike)
  itinerariesRouter.get('/:id/is-liked', itineraryController.checkIfLiked)
  itinerariesRouter.get('/:id/collaborators', itineraryController.getCollaborators)
  itinerariesRouter.post('/:id/collaborators', itineraryController.addCollaborator)
  itinerariesRouter.get('/:id/is-collaborator', itineraryController.checkIfCollaborator)

  return itinerariesRouter
}
```

Figura 20. Rutas del controlador de itinerarios

Algunas de estas operaciones son:

- GET /itineraries: Obtiene todos los itinerarios.
- POST /itineraries: Crea un nuevo itinerario.
- PATCH /itineraries/:id: Actualiza un itinerario existente.
- DELETE /itineraries/:id: Elimina un itinerario.

Los métodos de los controladores siguen el siguiente patrón común:

1. Recepción de la solicitud: los parámetros necesarios se obtienen de la solicitud HTTP, ya sea desde los parámetros de la URL (`req.params`), del cuerpo de la solicitud (`req.body`), o de los parámetros de consulta (`req.query`).
2. Validación de datos: antes de realizar cualquier operación, los parámetros se validan utilizando esquemas previamente definidos, para asegurar la integridad de los datos.
3. Autorización (si aplica): se comprueba que el usuario autenticado tenga permisos para realizar la operación solicitada.
4. Interacción con el modelo: una vez validados los datos, el controlador invoca el método correspondiente del modelo, que se encarga de ejecutar las operaciones SQL. Por ejemplo, si el controlador recibe una solicitud para crear un itinerario, invoca el método `create` de `ItineraryModel`.
5. Construcción de la respuesta: finalizada la operación, el controlador envía una respuesta HTTP utilizando el objeto `res`, que puede incluir un código de estado (200, 201, 404, etc.) y, si procede, datos en formato JSON.

El manejo de errores se realiza de manera centralizada. Si el modelo lanza una excepción o devuelve un resultado inesperado (por ejemplo, si no se encuentra un itinerario con el ID proporcionado), el controlador captura el error mediante bloques `try/catch` y lo reenvía al *middleware* de manejo de errores mediante `next(error)`. Este *middleware* se encarga de responder con un mensaje de error claro y un código de estado adecuado, como 400 (petición inválida), 404 (no encontrado) o 500 (error interno del servidor).

En la Figura 21 se presenta, a modo de ejemplo, el método `create` del `ItineraryController`, que utiliza internamente el modelo `ItineraryModel` para realizar operaciones CRUD relacionadas con itinerarios.

```
create = async (req, res, next) => {
  const { user } = req.session
  try {
    if (!user) throw new UnauthorizedError('Access not authorized')

    const result = validateItinerary(req.body)

    if (!result.success) {
      throw new InvalidInputError('Invalid itinerary data: ' + JSON.stringify(result.error.message))
    }
    if (user.id !== req.body.userId) {
      throw new UnauthorizedError('You are not authorized to create an itinerary for another user')
    }

    const itineraryId = await this.itineraryModel.create({ input: result.data })

    const locationUrl = `${req.protocol}://${req.get('host')}${req.originalUrl}/${itineraryId}`
    res.status(201).set('Location', locationUrl).end()
  } catch (error) {
    next(error)
  }
}
```

Figura 21. Método create del controlador de itinerarios

4.3.1.3 Autenticación

La autenticación de usuarios en el sistema se basa en el uso de **JSON Web Tokens (JWT)** [35], un estándar abierto que permite crear tokens compactos y seguros para la autenticación y autorización en aplicaciones web. Estos tokens, al estar firmados digitalmente, garantizan la integridad de la información que transmiten y permiten verificar que el remitente es quien dice ser. Los JWT son ampliamente utilizados debido a su eficiencia, seguridad y compatibilidad con diversas tecnologías. En la Figura 22 se puede observar un ejemplo de un token JWT y la decodificación de sus datos.

Encoded	Decoded
<pre>eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkNTU5MmZlM10zNTljLTExZjAtOTc2ZS04NjJyY2ZiMDYzNTIiLCJ1c2VybmFtZSI6Imp1YW5kYW5pZWwxNiIsIm1hdCI6MTc0NzgyNjI1MywzIHZhWjoxNzQ3ODI5ODUzfQ.OQeVWmmXqQSq--m7o0zeUgQV9iRkWKKQoYmLJDvF64g</pre>	HEADER: ALGORITHM & TOKEN TYPE <pre>{ "alg": "HS256", "typ": "JWT"}</pre> PAYLOAD: DATA <pre>{ "sub": "d5592fe2-359c-11f0-976e-862ccfb06352", "username": "juandaniel16", "iat": 1747826253, "exp": 1747829853}</pre>

Figura 22. Ejemplo de token JWT y su decodificación

Para gestionar la autenticación en el servidor, se ha empleado la biblioteca **jsonwebtoken** [36], que permite tanto generar como verificar los JWT. Los tokens se emiten una vez que el usuario se autentica correctamente proporcionando sus credenciales (nombre de usuario/email y contraseña). El proceso de autenticación se maneja en el controlador `AuthController`, donde se define el método de `login`, que se muestra en la Figura 23.

```
login = async (req, res, next) => {
  try {
    const result = validateLogin(req.body)
    if (!result.success) {
      throw new InvalidInputError('Invalid user data: ' + JSON.stringify(result.error.message))
    }

    const { usernameOrEmail, password } = result.data
    const isEmail = usernameOrEmail.includes('@')

    const user = isEmail
      ? await this.userModel.getByEmail({
          email: usernameOrEmail,
          includePassword: true
        })
      : await this.userModel.getByUsername({
          username: usernameOrEmail,
          includePassword: true
        })

    const passwordCorrect = user === null ? false : await bcrypt.compare(password, user.password_hash)
    if (!(user && passwordCorrect)) {
      throw new UnauthorizedError('Invalid user or password')
    }

    const tokenPayload = { id: user.id, username: user.username }
    const accessToken = jwt.sign(tokenPayload, JWT_SECRET, { expiresIn: '1h' })
    const refreshToken = jwt.sign(tokenPayload, JWT_SECRET, { expiresIn: '7d' })

    res
      .cookie('access_token', accessToken, {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: process.env.NODE_ENV === 'production' ? 'none' : 'strict',
        domain: process.env.NODE_ENV === 'production' ? '.travily.net' : undefined,
        maxAge: 1000 * 60 * 60
      })
      .cookie('refresh_token', refreshToken, {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: process.env.NODE_ENV === 'production' ? 'none' : 'strict',
        domain: process.env.NODE_ENV === 'production' ? '.travily.net' : undefined,
        path: '/auth/refresh-token',
        maxAge: 1000 * 60 * 60 * 24 * 7
      })
      .json({
        id: user.id,
        name: user.name,
        username: user.username,
        avatar: user.avatar,
        followers: user.followers,
        following: user.following
      })
  } catch (error) {
    next(error)
  }
}
```

Figura 23. Método login del controlador de autenticación

Al recibir una solicitud de inicio de sesión, el servidor comienza validando los datos de entrada proporcionados por el usuario. En caso de ser correctos, procede a verificar las credenciales utilizando la biblioteca **bcrypt** [37] para comparar la contraseña proporcionada con la almacenada de forma segura en la base de datos. Si la autenticación es exitosa, se genera un token JWT, que contiene un conjunto de "claims" (declaraciones) personalizadas, como el identificador (*sub*) y nombre de usuario, así como la fecha de emisión (*iat*) y expiración (*exp*) del token. Este token se firma utilizando el algoritmo *HS256* y una clave secreta, garantizando que no pueda ser alterado sin invalidar su integridad.

El token generado se envía al cliente como parte de la respuesta, incluyéndolo como una cookie `access_token`, que se utiliza para autenticar las solicitudes posteriores. Esta cookie tiene el atributo *HttpOnly*, lo que la hace inaccesible desde JavaScript en el lado del cliente,

mejorando la seguridad frente a ataques de tipo XSS (del inglés “*Cross-Site Scripting*”) [38]. Por este motivo, se ha decidido almacenar el token en una cookie en lugar de utilizar la cabecera *Authorization* con un token *Bearer*. Además, la cookie se establece con la política *SameSite* en *none*, se marca como *secure* y se asocia al dominio *.travily.net*, lo que refuerza la seguridad en entornos donde el *front-end* y el *back-end* están desplegados en dominios distintos. La duración de la cookie es de 1 hora, acorde con el tiempo de expiración del token.

Junto al *access_token*, también se genera un token de refresco, llamado *refresh_token*, con una duración mayor (7 días), que se incluye en la respuesta como una cookie. Cuando el *access_token* ha expirado, el cliente puede enviar el *refresh_token* al servidor para obtener un nuevo *access_token* sin necesidad de que el usuario vuelva a autenticarse. Este proceso es gestionado por el método *refreshToken* en el controlador de autenticación, tal como se muestra en la Figura 24.

```
refreshToken = async (req, res, next) => {
  try {
    const refreshToken = req.cookies.refresh_token

    const data = jwt.verify(refreshToken, JWT_SECRET)

    const tokenPayload = { sub: data.sub, username: data.username }
    const newAccessToken = jwt.sign(tokenPayload, JWT_SECRET, { expiresIn: '1h' })

    res
      .cookie('access_token', newAccessToken, {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: process.env.NODE_ENV === 'production' ? 'none' : 'strict',
        domain: process.env.NODE_ENV === 'production' ? '.travily.net' : undefined,
        maxAge: 1000 * 60 * 60
      })
      .status(204)
      .end()
  } catch (error) {
    next(error)
  }
}
```

Figura 24. Método *refreshToken* del controlador de autenticación

El cliente debe incluir el *access_token* en las cookies de todas las solicitudes posteriores al servidor. Estas son interceptadas por un *middleware* de autenticación, el cual se puede observar en la Figura 25, que se encarga de verificar la validez del token en cada petición.

```
export const auth = (req, res, next) => {
  const accessToken = req.cookies.access_token
  req.session = { user: null }

  try {
    if (accessToken) {
      const data = jwt.verify(accessToken, JWT_SECRET)
      req.session.user = data
    }
  } catch {}

  next()
}
```

Figura 25. *Middleware* de autenticación

Este *middleware* extrae el token de la cookie *access_token* y lo valida utilizando la biblioteca *jsonwebtoken*. Si el token es válido y no ha expirado, se extrae la información del usuario del token y se almacena en *req.session.user*, lo que permite acceder a los datos del usuario

en los controladores correspondientes. En caso contrario, el valor de `req.session.user` permanece como `null`.

En los controladores que requieren autenticación se comprueba si el usuario está presente en la sesión. Si no lo está, se devuelve una respuesta con el código de estado HTTP 401 (*Unauthorized*). Esto puede deberse a errores como *TokenExpiredError* o *JsonWebTokenError*, que indican que el token ha expirado o no es válido, y que por tanto debe renovarse o volver a autenticarse, respectivamente.

En cuanto a la seguridad de las contraseñas, estas nunca se almacenan en texto plano en la base de datos. Durante el proceso de registro, la contraseña del usuario se encripta utilizando la biblioteca **bcrypt** antes de ser almacenada. Al realizar el `login`, el servidor utiliza `bcrypt.compare()` para verificar que la contraseña proporcionada por el usuario coincida con la versión almacenada de la misma.

La arquitectura adoptada es **sin estado** (*stateless*), lo que significa que el servidor no almacena información sobre la sesión del usuario. En cambio, el token JWT se utiliza como única fuente de autenticación, permitiendo que la aplicación sea escalable y no dependa de sesiones persistentes en el servidor.

Finalmente, cuando el usuario decide cerrar sesión, se eliminan las cookies correspondientes al `access_token` y al `refresh_token`, asegurando que no queden datos de sesión activos.

4.3.1.4 WebSockets

En Travily, la edición colaborativa de itinerarios en tiempo real es posible gracias a la integración de **WebSockets** [10] mediante la biblioteca **Socket.IO** [9]. Esta tecnología permite la comunicación bidireccional entre el servidor y los clientes, lo que facilita que los usuarios colaboren de manera simultánea y vean los cambios en los itinerarios al instante, sin necesidad de recargar la página.

El archivo `websocket.js`, ubicado en la carpeta `/src/websocket`, contiene la configuración y lógica de esta comunicación en tiempo real. A continuación, se detalla su funcionamiento. En primer lugar, se establece un servidor *Socket.IO* asociado al servidor HTTP de la aplicación, configurado con opciones de CORS para asegurar que solo los orígenes permitidos puedan acceder a los WebSockets. La Figura 26 muestra la configuración de este servidor.

```
export const initializeWebSocket = ({ server, dayModel, eventModel }) => {  
  const io = new Server(server, {  
    cors: {  
      origin: ACCEPTED_ORIGINS,  
      methods: ['GET', 'POST']  
    }  
  })  
}
```

Figura 26. Configuración del servidor de WebSockets con *Socket.IO*

Cuando un usuario se conecta, el servidor le asigna una "sala" correspondiente al ID del itinerario que está editando. De esta forma, solo los usuarios que estén trabajando en el mismo itinerario reciben las actualizaciones en tiempo real de las acciones que realicen otros colaboradores. Además, antes de permitir que un nuevo usuario cargue un itinerario al unirse a una sala, el servidor se asegura de persistir cualquier cambio pendiente para garantizar que

dicho usuario reciba una versión actualizada del contenido, manteniendo la consistencia y la integridad de los datos. Esto se puede observar en la Figura 27.

```
io.on('connection', (socket) => {
  console.log('Un nuevo cliente se ha conectado')

  // El cliente se une a la sala del itinerario que está editando
  socket.on('join-itinerary', async (itineraryId) => {
    socket.join(itineraryId)
    console.log(`El cliente ${socket.id} se ha unido a la sala del itinerario: ${itineraryId}`)

    try {
      // Persistir datos pendientes antes de permitir la carga
      if (itineraryUpdates[itineraryId]) {
        const { reordered } = itineraryUpdates[itineraryId]

        if (reordered) {
          await eventController.reorderEvents({
            itineraryId,
            dayId: reordered.dayId,
            events: reordered.events
          })
        }

        delete itineraryUpdates[itineraryId]
      }

      // Avisar al cliente que la persistencia ha terminado
      socket.emit('itinerary-ready')
    } catch (error) {
      console.error('Error al persistir datos antes de cargar el itinerario:', error)
    }
  })
})
```

Figura 27. Unión a una sala y persistencia de cambios pendientes

Las modificaciones realizadas por los usuarios, como la edición de información de los itinerarios, la adición, edición o eliminación de días y eventos, y el reordenamiento de eventos, son enviadas a todos los clientes conectados a la misma sala. Esto asegura que todos los colaboradores vean los cambios al instante. Además, estas acciones se persisten en la base de datos, garantizando que la información se mantenga actualizada y disponible incluso después de finalizar la sesión. A modo de ejemplo, en la Figura 28 se muestra la lógica para la edición de un evento y su propagación a otros colaboradores en tiempo real.

```
socket.on('edit-event', async ({ itineraryId, dayId, eventId, updatedEventData }) => {
  try {
    await eventController.updateEvent({ eventId, updatedEventData })
    io.to(itineraryId).emit(`itinerary-update-${itineraryId}`, {
      action: 'edit-event',
      dayId,
      eventId,
      updatedEventData
    })
  } catch (error) {
    console.error(`${error.name}: ${error.message}`)
    socket.emit('error', { message: error.message })
  }
})
```

Figura 28. Propagación en tiempo real de la edición de un evento

Cabe destacar que la operación de reordenar eventos es la única que no se persiste inmediatamente. En su lugar, el servidor almacena temporalmente los nuevos órdenes, los propaga a todos los participantes en tiempo real y los guarda en la base de datos de forma

periódica, como se observa en la Figura 29. Esta estrategia mejora la eficiencia y evita una sobrecarga innecesaria en el sistema.

```
setInterval(async () => {
  const itineraryIdsWithUpdates = Object.keys(itineraryUpdates).filter(
    (itineraryId) => itineraryUpdates[itineraryId]?.reordered
  )
  if (itineraryIdsWithUpdates.length === 0) return

  try {
    await Promise.all(
      itineraryIdsWithUpdates.map(async (itineraryId) => {
        const { reordered } = itineraryUpdates[itineraryId]

        if (reordered) {
          try {
            await eventController.reorderEvents({
              itineraryId,
              dayId: reordered.dayId,
              events: reordered.events
            })
          } catch (error) {
            console.error(`${error.name}: ${error.message}`)
            io.to(itineraryId).emit('error', { message: error.message })
          }
        }

        delete itineraryUpdates[itineraryId]
      })
    )
  } catch (error) {
    console.error(`${error.name}: ${error.message}`)
  }
}, UPDATE_INTERVAL)
```

Figura 29. Persistencia periódica de cambios

Finalmente, cuando un usuario se desconecta, el servidor lo elimina de la sala del itinerario, garantizando que no reciba más actualizaciones. Además, el usuario puede abandonar la sala del itinerario de manera explícita en cualquier momento. Esto se muestra en la Figura 30.

```
// El cliente abandona la sala del itinerario que está editando
socket.on('leave-itinerary', (itineraryId) => {
  socket.leave(itineraryId)
  console.log(`El cliente ${socket.id} ha abandonado la sala del itinerario: ${itineraryId}`)
})

socket.on('disconnect', () => {
  console.log('Un cliente se ha desconectado')
})
```

Figura 30. Desconexión de un usuario y abandono de una sala

4.3.1.5 Gestión de la carga de archivos

La carga de archivos es una funcionalidad común en aplicaciones web modernas, especialmente en aquellas que permiten personalizar el perfil del usuario o asociar imágenes a determinados elementos del sistema. En esta aplicación, los usuarios pueden subir imágenes para cuatro tipos de recursos: su avatar, un itinerario, un evento o una lista.

Inicialmente, las imágenes se almacenaban en carpetas del servidor. Sin embargo, esta lógica fue reemplazada por una solución basada en la nube utilizando el servicio **Cloudinary** [39], mejorando la escalabilidad, disponibilidad y gestión de las imágenes. Para integrar este servicio se ha utilizado la biblioteca **multer-storage-cloudinary** [40], en combinación con **multer** [41] como *middleware* para procesar formularios *multipart/form-data*, que es el tipo de contenido habitual en formularios de subida de archivos.

El almacenamiento ha sido configurado con parámetros dinámicos en función de la URL de la solicitud. Tal como se muestra en la Figura 31, cada tipo de imagen se almacena en una carpeta distinta dentro del espacio de Cloudinary:

- `avatars`: para los avatares de usuario.
- `itineraries`: para las imágenes asociadas a itinerarios.
- `events`: para las imágenes de eventos.
- `lists`: para las imágenes de listas.

Además, se aplica una transformación de tamaño a cada imagen en el momento de la subida, con el fin de optimizar el almacenamiento y mejorar el rendimiento en el cliente. Por ejemplo, los avatares se redimensionan a 200x200 píxeles, mientras que las imágenes de itinerarios y listas se limitan a 1000x700 píxeles. El nombre de cada archivo se genera a partir del nombre de usuario seguido del *timestamp* actual, lo que evita colisiones.

```
const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: (req, file) => {
    let folder = 'others'
    let transformation = [{ width: 500, height: 500, crop: 'limit' }]

    if (req.url.includes('/avatar')) {
      folder = 'avatars'
      transformation = [{ width: 200, height: 200, crop: 'limit' }]
    } else if (req.url.includes('/itinerary-image')) {
      folder = 'itineraries'
      transformation = [{ width: 1000, height: 700, crop: 'limit' }]
    } else if (req.url.includes('/event-image')) {
      folder = 'events'
      transformation = [{ width: 500, height: 500, crop: 'limit' }]
    } else if (req.url.includes('/list-image')) {
      folder = 'lists'
      transformation = [{ width: 1000, height: 700, crop: 'limit' }]
    }

    return {
      folder,
      allowed_formats: ['jpg', 'jpeg', 'png'],
      transformation,
      public_id: file.originalname
    }
  }
})

export const upload = multer({ storage })
```

Figura 31. Configuración del almacenamiento en Cloudinary

Cada tipo de carga tiene su propia ruta definida dentro del *router* `/upload`, como por ejemplo `/upload/avatar` o `/upload/event-image`. Estas rutas procesan el archivo enviado y devuelven directamente la URL de la imagen subida a Cloudinary, accesible desde el cliente. En la Figura 32 se muestra, a modo de ejemplo, la implementación de la ruta de subida de un avatar de usuario.

```
uploadRouter.post('/avatar', upload.single('file'), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: 'No file uploaded' })
  }
  const avatarUrl = req.file.path
  res.json({ avatarUrl })
})
```

Figura 32. Ruta de subida de un avatar de usuario

4.3.2 Implementación del *front-end*

El *front-end* de la aplicación representa la parte visible y accesible para los usuarios, gestionando la interacción con el sistema y el intercambio de datos con el *back-end* mediante peticiones HTTP. Para su desarrollo, se han empleado tecnologías modernas que garantizan una experiencia de usuario rápida, fluida y adaptada a diferentes dispositivos.

La estructura y funcionalidad de la interfaz se han desarrollado con **React**, una biblioteca de JavaScript basada en componentes reutilizables y modulares, junto con **TypeScript**, que facilita la detección de errores mediante tipado estático. En cuanto al diseño, se ha utilizado **Tailwind CSS**, un *framework* que permite aplicar estilos directamente mediante clases en los componentes. Además, se ha integrado **Mantine**, una biblioteca de UI basada en Tailwind CSS que ofrece componentes de React personalizables y adaptables a cualquier dispositivo.

La comunicación con el servidor a través de solicitudes HTTP se realiza mediante la **API Fetch** [42] nativa de JavaScript. Por otro lado, la navegación entre las distintas páginas de la aplicación se gestiona mediante la biblioteca **React Router** [43], lo que permite definir rutas y asociarlas a los componentes correspondientes de manera sencilla.

Además, el entorno de desarrollo se ha configurado con **Vite** [44], una herramienta que acelera la creación y el arranque de proyectos React, mejorando los tiempos de carga y el rendimiento. También se utiliza **JSX** [45], una extensión de JavaScript que permite escribir HTML dentro del código, simplificando la creación de componentes de la interfaz de usuario.

En los siguientes apartados se presenta la estructura del proyecto y se describen algunos de los componentes más relevantes implementados, destacando aquellos que presentan una mayor complejidad o una funcionalidad especialmente importante dentro del sistema.

4.3.2.1 Estructura

La estructura del *front-end* de la aplicación, mostrada en la Figura 33, se organiza principalmente en dos directorios: `public` y `src`.

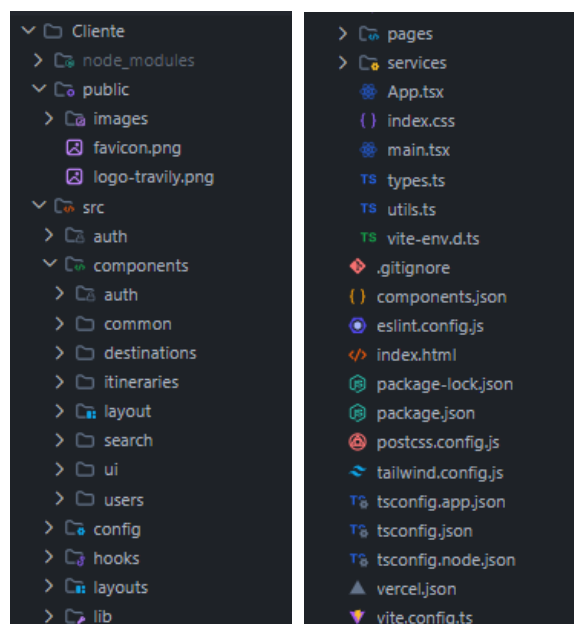


Figura 33. Estructura de carpetas del *front-end* de Travily

El directorio `public` contiene recursos estáticos que no requieren procesamiento previo, como el icono de la web (`favicon.png`), el logo (`logo-travily.png`) y otras imágenes utilizadas en distintas partes de la aplicación, organizadas en la carpeta `images`.

El archivo `index.html`, ubicado en la raíz del proyecto, actúa como el punto de entrada HTML de la aplicación. En él, se enlazan los recursos generados tras la compilación y se define el elemento raíz con el identificador `root`, sobre el que React montará el contenido dinámico. En la raíz del proyecto también se encuentran varios archivos de configuración importantes, como el archivo `package.json`, que gestiona las dependencias y scripts del proyecto, y los archivos de configuración de Tailwind, Vite y TypeScript, entre otros.

Por otro lado, el directorio `src` alberga todo el código fuente de la aplicación y está organizado de manera modular para facilitar su mantenimiento y escalabilidad. Dentro de `src` se encuentran carpetas especializadas como `components`, donde se almacenan los componentes de React que representan bloques reutilizables de la interfaz de usuario, y `pages`, que contiene los componentes de página correspondientes a cada ruta principal de la aplicación. La carpeta `hooks` agrupa `hooks` personalizados que encapsulan lógica reutilizable, mientras que `layouts` alberga componentes de estructura general de las páginas, `services` contiene funciones para interactuar con la API, `auth` gestiona la lógica de autenticación y `config` contiene configuraciones globales del proyecto. Además, el archivo `utils.ts` agrupa funciones y constantes reutilizables, mientras que `types.ts` define los tipos de TypeScript utilizados para estructurar los datos y garantizar la consistencia del sistema.

En cuanto a los archivos principales de `src`, el archivo `main.tsx` (Figura 34) es el punto de entrada de la aplicación React. En él, se crea la raíz de la aplicación utilizando `createRoot` de `ReactDOM`, que monta el componente principal `App` en el DOM del navegador, específicamente dentro del elemento con id `root` definido en el `index.html`. Todo esto se encapsula bajo `React.StrictMode` para facilitar la detección de posibles errores durante el desarrollo. Además, en este archivo se configura el proveedor de React Router, envolviendo toda la aplicación en el `BrowserRouter`, lo que permite gestionar la navegación interna. También se configura `AuthProvider` para manejar el estado de autenticación global de la aplicación, y se incluye el archivo `index.css`, que importa las directivas principales de Tailwind CSS y define estilos base personalizados, incluyendo variables CSS para los temas claro y oscuro.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.tsx'
import './index.css'
import '@mantine/core/styles.css'
import '@mantine/carousel/styles.css'
import { BrowserRouter } from 'react-router'
import { AuthProvider } from './auth/AuthProvider.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <AuthProvider>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </AuthProvider>
  </StrictMode>
)
```

Figura 34. Archivo `main.tsx`

El componente `App.tsx` (Figura 35), por su parte, constituye la base de la estructura funcional de la aplicación. Dentro de este componente se configura el proveedor de Mantine, la biblioteca de componentes UI utilizada en el proyecto, y se definen las rutas principales mediante la biblioteca React Router, permitiendo que la aplicación se comporte como una SPA (del inglés “*Single Page Application*”). El componente `Routes` envuelve la lista de rutas individuales, cada una declarada con un `Route` que especifica el componente que debe renderizarse según la URL actual.

```
function App() {
  return (
    <MantineProvider
      defaultColorScheme="light"
      theme={{
        primaryColor: 'brand',
        primaryShade: 5,
        colors: {
          brand: ['#e6fcf5', '#c3fae8', '#96f2d7', '#63e6be', '#38d9a9', '#20c997', '#12b886', '#0ca678', '#099268', '#087f5b']
        },
        breakpoints: {
          xxs: '30rem', xs: '35rem', sm: '40rem',
          md: '48rem', lg: '64rem', lg72rem: '72rem', xl: '80rem'
        }
      }}
    >
      <ScrollToTop />
      <Routes>
        <Route element={<MainLayout />} />
        <Route path="/" element={<Home />} />
        <Route path="/:username" element={<Profile />} />
        <Route path="/:username/followers" element={<Followers />} />
        <Route path="/:username/following" element={<Following />} />
        </Route>
        <Route path="/:username/itineraries" element={<UserItineraries />} />
        <Route path="/:username/favorites" element={<UserItineraries />} />
        <Route path="/:username/lists" element={<UserItineraryLists />} />
        <Route path="/:username/itineraries/:itineraryId" element={<Itinerary />} />
        <Route path="/lists/:listId" element={<ItineraryList />} />
        <Route element={<ProtectedRoute />} />
        <Route path="/create-itinerary" element={<ItineraryForm />} />
        </Route>
        <Route element={<ProtectedRoute />} />
        <Route path="/create-list" element={<ItineraryListForm />} />
        </Route>
        <Route element={<ProtectedRoute />} />
        <Route path="/connect" element={<Connect />} />
        </Route>
        <Route element={<ProtectedRoute />} />
        <Route path="/friends" element={<Friends />} />
        </Route>
        <Route element={<ProtectedRoute />} />
        <Route path="/friends/itineraries" element={<FriendsItineraries />} />
        </Route>
        <Route element={<ProtectedRoute />} />
        <Route path="/friends/lists" element={<FriendsLists />} />
        </Route>
        <Route element={<ProtectedRoute />} />
        <Route path="/settings" element={<ProfileSettings />} />
        </Route>
        <Route path="/search" element={<SearchResults />} />
        </Route>
        <Route element={<AuthLayout />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/about-us" element={<AboutUs />} />
        <Route path="/contact-us" element={<ContactUs />} />
        <Route path="/privacy-policy" element={<PrivacyPolicy />} />
        <Route path="/terms-of-use" element={<TermsOfUse />} />
        <Route path="*" element={<NotFound />} />
        </Route>
      </Routes>
    </MantineProvider>
  )
}
```

Figura 35. Componente `App.tsx`

Algunas páginas están protegidas, de modo que solo los usuarios autenticados pueden acceder a ellas. Para gestionar esta protección, se utiliza el componente `ProtectedRoute` (Figura 36), ubicado en `components/utils/`. Este componente utiliza el *hook* personalizado `useAuth` para verificar si el usuario está autenticado. Si lo está, `ProtectedRoute` renderiza el elemento hijo mediante el componente `Outlet`. Si no, redirige a la página de inicio de sesión.

```
export const ProtectedRoute = () => {  
  const { user, isLoading } = useAuth()  
  const location = useLocation()  
  
  if (isLoading) return  
  if (!user) {  
    return <Navigate to="/login" state={{ from: location }} />  
  }  
  
  return <Outlet />  
}
```

Figura 36. Componente `ProtectedRoute`

Finalmente, las rutas se agrupan bajo componentes de *layout* para asegurar una estructura coherente en las páginas de la aplicación. Un ejemplo de esto es `MainLayout`, que envuelve las rutas principales y proporciona elementos de navegación que permanecen constantes en todas las páginas, como el encabezado (`Header`), el pie de página (`Footer`), la barra de navegación inferior en dispositivos móviles y las columnas laterales en la versión de escritorio.

4.3.2.2 Página de inicio de sesión

La página de inicio de sesión, mostrada en la Figura 37, permite a los usuarios registrados acceder a la aplicación. Este componente forma parte del *layout* `AuthLayout`, que mantiene el `Header` y el `Footer` constantes en todas las páginas de autenticación.

Figura 37. Página de inicio de sesión

El formulario consta de dos campos: uno para el nombre de usuario o correo electrónico y otro para la contraseña, como se muestra en la Figura 38. Ambos campos utilizan el *hook* `useState` [46] de React para manejar el estado. En la Figura 39 se puede observar la declaración de estos estados: `usernameOrEmail`, `password` y `error`.

```
<form onSubmit={handleSubmit} className='mb-4'>
  <TextInput
    label='Usuario/Correo electrónico'
    value={usernameOrEmail}
    size='md'
    onChange={(e) => setUsernameOrEmail(e.target.value)}
    required
    withAsterisk={false}
  />
  <PasswordInput
    label='Contraseña'
    value={password}
    size='md'
    onChange={(e) => setPassword(e.target.value)}
    required
    withAsterisk={false}
    mt='sm'
  />
  <Button
    type='submit'
    loading={loading}
    loaderProps={{ type: 'dots' }}
    fullWidth
    color='brand'
    mt='xl'
  >
    Entrar
  </Button>
</form>
```

Figura 38. Formulario de inicio de sesión

```
const [usernameOrEmail, setUsernameOrEmail] = useState('')
const [password, setPassword] = useState('')
const [error, setError] = useState('')
```

Figura 39. Declaración de los estados para el formulario de inicio de sesión

Cada input del formulario tiene un evento `onChange` que actualiza el valor del estado correspondiente, garantizando que los datos introducidos por el usuario estén sincronizados con el estado interno del componente. Además, si se produce un error al iniciar sesión (como `InvalidInputError`, `UnauthorizedError` o `NotFoundError`), se muestra un mensaje de error personalizado mediante la variable `error`, gestionada también con `useState`.

Cuando el usuario envía el formulario, se ejecuta la función `handleSubmit` (Figura 40), que se encarga de realizar la solicitud de autenticación a través del *hook* `useAuth`. Este *hook* encapsula toda la lógica de autenticación y se apoya en un contexto global (`AuthContext`), proporcionado por el componente `AuthProvider`, que envuelve la aplicación y expone métodos como `login`, `logout`, `register` o `refreshUser`, así como el estado del usuario (`user`) y un indicador de carga (`isLoading`).


```
const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault()
  setError('')
  setLoading(true)

  try {
    await login(usernameOrEmail, password)
    if (onLoginSuccess) {
      onLoginSuccess?.()
    } else {
      navigate(location.state?.from?.pathname || '/')
    }
  } catch (error) {
    setLoading(false)
    if (error instanceof Error) {
      switch (error.message) {
        case 'InvalidInputError':
          setError('Datos de entrada inválidos. Asegúrate de que todos los campos estén correctos.')
          break
        case 'UnauthorizedError':
          setError('Usuario o contraseña incorrectos. Por favor, inténtalo de nuevo.')
          break
        case 'NotFoundError':
          setError('Usuario o contraseña incorrectos. Por favor, inténtalo de nuevo.')
          break
        default:
          setError('Ocurrió un error inesperado. Por favor, inténtalo de nuevo.')
      }
    } else {
      setError('Ocurrió un error inesperado. Por favor, inténtalo de nuevo.')
    }
  }
}
```

Figura 40. Función handleSubmit

El método login (Figura 41), definido en AuthProvider, delega la solicitud al servicio authService (Figura 42), el cual realiza la llamada a la API con las credenciales proporcionadas. Si la autenticación es exitosa, el usuario es redirigido a la página principal utilizando el *hook* useNavigate [47] de React Router.

```
const login = async (usernameOrEmail: string, password: string) => {
  const userData = await authService.login(usernameOrEmail, password)
  setUser(userData)
  localStorage.setItem('userId', userData.id)
}
```

Figura 41. Método login en AuthProvider

```
login: async (usernameOrEmail: string, password: string) => {
  const response = await fetch(`${API_BASE_URL}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify({ usernameOrEmail, password })
  })

  if (!response.ok) {
    const errorData = await response.json()
    throw new Error(errorData.error || 'Login error')
  }

  return await response.json()
},
```

Figura 42. Método login en authService

4.3.2.3 Página principal

La página principal de Travily, mostrada en la Figura 43, actúa como punto de entrada a la aplicación tras una autenticación exitosa. Esta corresponde al componente Home, el cual forma parte del *layout* general MainLayout. Dicho *layout* incluye elementos constantes como el Header y el Footer, y en dispositivos con pantalla grande (versión de escritorio), define también la columna izquierda (SideBar), que ofrece accesos directos de navegación, y la

columna derecha (RightColumn), que muestra sugerencias de usuarios a seguir. Ambas columnas permanecen fijas en todas las páginas principales de la aplicación.

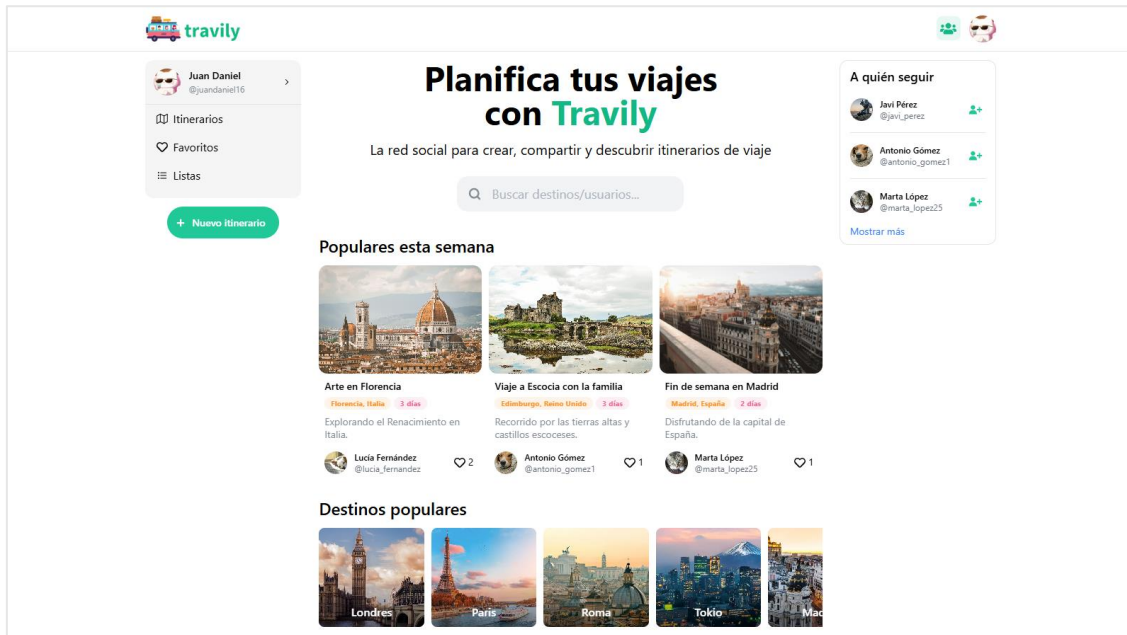


Figura 43. Página principal (versión escritorio)

En dispositivos móviles, la estructura de columnas se sustituye por una única columna central y una barra de navegación inferior (BottomNavBar) que permite acceder rápidamente a las secciones principales ('Inicio', 'Explorar', 'Crear nuevo itinerario', 'Amigos' y 'Mi perfil'), como se muestra en la Figura 44.

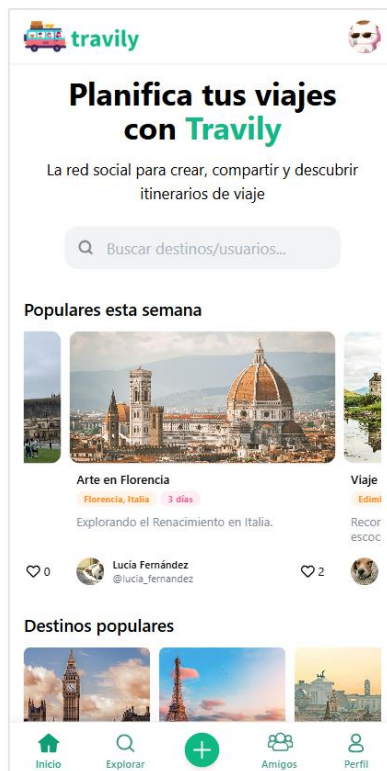


Figura 44. Página principal (versión móvil)

El componente `Home` hace uso del contexto global de autenticación (`AuthContext`) mediante el *hook* `useAuth` para acceder a los datos del usuario actual. A partir de esa información, el componente adapta su contenido a través del uso de condicionales en la interfaz. También utiliza *hooks* de estado como `useState` y `useEffect` para gestionar la carga de datos asíncrona. En concreto, al montar el componente o al cambiar el usuario, se ejecuta un `useEffect` que invoca dos funciones: `fetchPopularItineraries` y `fetchFriendsItineraries`, encargadas de obtener los itinerarios populares y los creados recientemente por los amigos del usuario. Ambas funciones, mostradas en la Figura 45, realizan peticiones a la API a través del servicio `itineraryService` y gestionan sus propios estados de carga (`loadingItineraries` y `loadingFriendsItineraries`). Los datos obtenidos se almacenan en los estados `itineraries` y `friendsItineraries`, respectivamente.

```
const fetchPopularItineraries = async () => {
  try {
    const itineraries = await itineraryService.getAll({ sort: 'popular' })
    setItineraries(itineraries)
  } catch {
    console.error('Error fetching popular itineraries')
    setError(true)
  } finally {
    setTimeout(() => {
      setLoadingItineraries(false)
    }, 250)
  }
}

const fetchFriendsItineraries = async () => {
  if (user) {
    try {
      const itineraries = await itineraryService.getAll({
        sort: 'newest',
        followedBy: user.id
      })
      setFriendsItineraries(itineraries)
    } catch {
      console.error('Error fetching itineraries from followed users')
      setError(true)
    } finally {
      setTimeout(() => {
        setLoadingFriendsItineraries(false)
      }, 250)
    }
  }
}
```

Figura 45. Funciones `fetchPopularItineraries` y `fetchFriendsItineraries`

El componente muestra un mensaje de bienvenida seguido de una barra de búsqueda (`SearchInput`), que permite localizar tanto itinerarios en destinos concretos como perfiles de otros usuarios. A continuación, se dispone un carrusel con los itinerarios más populares de la semana. Mientras se obtienen los datos, se muestran esqueletos de carga (`Skeleton`) para mantener la coherencia visual y evitar saltos de contenido. Si ocurre un error o no hay itinerarios, se muestra un mensaje de error o de ausencia de contenido. Después, se incluye un carrusel con los destinos más populares, mediante el componente `PopularDestinations`. Por último, en caso de que el usuario tenga contactos seguidos, se presenta una sección adicional donde se muestran en otro carrusel los itinerarios más recientes de sus amigos.

El renderizado dinámico de las tarjetas de itinerario se realiza dentro del componente `ItinerariesCarousel`, mostrado en la Figura 46, al cual se le pasan los datos como `prop`. Este componente itera sobre la lista de itinerarios utilizando el método `map`, y por cada elemento genera un componente `ItineraryCard`, al que se le pasa el itinerario correspondiente como `prop`.

```
export const ItinerariesCarousel = ({ itineraries }: { itineraries: ItinerarySimpleType[] }) => {
  return (
    <Carousel
      height={295}
      slideSize={{
        base: '75%',
        xxs: '65%',
        xs: '55%',
        sm: '40%',
        md: '34%',
        lg: '27%',
        lg72rem: '51%',
        xl: '33.33333%'
      }}
      slideGap='xs'
      align='center'
      slidesToScroll='auto'
      withControls={{window.innerWidth > 640}}
      dragFree
      loop
      className='group'
      classNames={{
        control:
          '!opacity-0 group-hover:!opacity-100 !transition-opacity !duration-300 -mt-4 -mx-1 w-7 h-7'
      }}
    >
      {itineraries.map((itinerary) => (
        <Carousel.Slide key={itinerary.id}>
          <div className='flex h-full overflow-hidden rounded-xl'>
            <ItineraryCard itinerary={itinerary} />
          </div>
        </Carousel.Slide>
      ))}
    </Carousel>
  )
}
```

Figura 46. Componente `ItinerariesCarousel`

En la Figura 47 se puede observar el componente `ItineraryCard`, que recibe como propiedad un objeto de tipo `ItinerarySimpleType` y se encarga de mostrar visualmente la información de cada itinerario. Internamente, utiliza el *hook* `useEffect` para obtener los datos públicos del creador del itinerario, haciendo una petición asíncrona al método `getById` del servicio `userService`, con el `userId` del itinerario. Estos datos se almacenan en el estado `userData` mediante el *hook* `useState`.

```

interface ItineraryCardProps { itinerary: ItinerarySimpleType }

export const ItineraryCard = ({ itinerary }: ItineraryCardProps) => {
  const [userData, setUserData] = useState<UserPublic | null>(null)

  useEffect(() => {
    const fetchUserData = async () => {
      try {
        const data = await userService.getById(itinerary.userId)
        setUserData(data)
      } catch {
        console.error('Error fetching user data')
      }
    }
    fetchUserData()
  }, [itinerary.userId])

  return (
    <div className='flex flex-col flex-grow rounded-xl'>
      <div className='h-[52%] min-h-[52%] rounded-xl overflow-hidden'>
        <Link to={`/itineraries/${itinerary.id}`}>
          <Image
            src={itinerary.image || '/images/placeholder/landscape-placeholder.svg'}
            h='100%'
            alt={itinerary.title}
            className='transition duration-300 transform hover:scale-105'
          />
        </Link>
      </div>
      <div className='flex flex-col justify-between flex-grow px-2'>
        <div className='flex flex-col gap-1 my-2'>
          <Link to={`/itineraries/${itinerary.id}`}>
            <Text size='sm' fw={500} lineClamp={1}>{itinerary.title}</Text>
          </Link>
          <div className='flex gap-1 mb-1'>
            <Badge variant='light' color='orange' size='sm' className='!normal-case'>
              {itinerary.location.countryName}
              ? itinerary.location.name === itinerary.location.countryName
              ? itinerary.location.name
              : `${itinerary.location.name}, ${itinerary.location.countryName}`
              : itinerary.location.name
            </Badge>
            <Badge variant='light' color='pink' size='sm' className='!normal-case'>
              {calculateTotalDays(itinerary.startDate, itinerary.endDate)} días
            </Badge>
          </div>
          <div>
            <Text fz='sm' c='dimmed' lh={1.3} lineClamp={2}>{itinerary.description}</Text>
          </div>
          <Group justify='space-between'>
            <div className='flex items-center w-[75%] overflow-hidden'>
              <Center className='overflow-hidden'>
                <Link to={`/${userData?.username}`}>
                  <Avatar
                    src={userData?.avatar || '/images/placeholder/avatar-placeholder.svg'}
                    mr='xs'
                    size={32}
                  />
                </Link>
                <div className='leading-none overflow-hidden'>
                  <Link to={`/${userData?.username}`}>
                    <Text truncate='end' className='!font-medium !text-xs'>{userData?.name}</Text>
                    <Text truncate='end' className='!text-gray-500 !text-xs'>@{userData?.username}</Text>
                  </Link>
                </div>
              </Center>
            </div>
            <LikeButton itinerary={itinerary} />
          </Group>
        </div>
      </div>
    </div>
  )
}

```

Figura 47. Componente ItineraryCard

En cuanto a su estructura, la tarjeta está dividida en dos secciones principales. La parte superior muestra la imagen de portada del itinerario, que al hacer clic redirige a la página de detalles. Esta redirección se realiza mediante el componente `Link` de React Router, el cual permite actualizar la URL y renderizar el componente correspondiente sin recargar la página, mejorando la fluidez de la aplicación. La parte inferior incluye el título, el destino, la duración, una descripción abreviada y la información del creador: nombre, *username* y avatar, que enlazan a su perfil público. Finalmente, también se incluye un botón interactivo para marcar el itinerario como favorito. Todo el diseño es responsive y se adapta a diferentes resoluciones de pantalla, garantizando una navegación fluida y consistente.

4.3.2.4 Página de detalles de un itinerario

La página de detalles de un itinerario, mostrada en la Figura 48, corresponde al componente `Itinerary`, que constituye una de las partes más complejas e interactivas de la aplicación. Este componente combina visualización, edición, sincronización en tiempo real entre múltiples usuarios, formularios en diversos modales y subida de archivos. La página está disponible para cualquier usuario, esté o no registrado, siempre que el itinerario sea público. Si el itinerario es privado, solo pueden acceder quienes sean colaboradores. Existen dos modos de visualización según la relación del usuario con el itinerario: modo solo lectura (visualización pública) y modo edición (cuando el usuario es creador o colaborador).

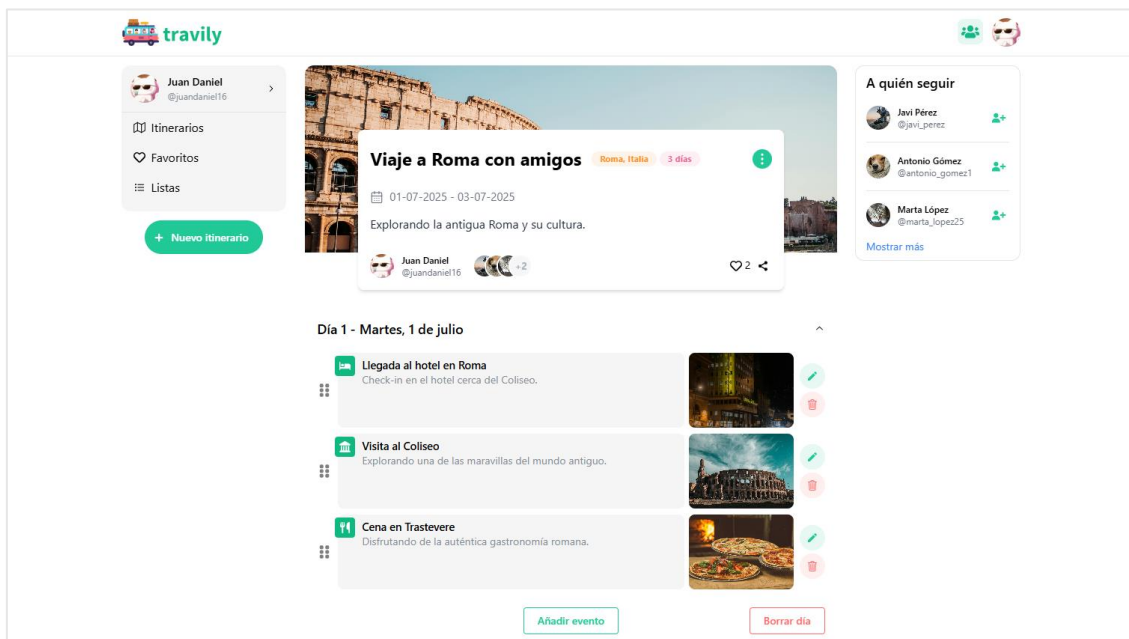


Figura 48. Página de detalles de un itinerario en modo edición

El componente se estructura en tres bloques principales. En primer lugar, en la parte superior se encuentra la cabecera con la información general del itinerario, que incluye la imagen principal (editable en modo edición), el título (editable *in-place*), la localización, la duración del viaje en días, las fechas de inicio y fin (con selectores de calendario en modo edición), y una descripción también editable. En esta sección también se muestra el creador del itinerario junto a su avatar, así como los avatares de los colaboradores, que pueden desplegarse para mostrar la lista completa. Además, se incluyen los botones para dar "me gusta" (`LikeButton`) y compartir (`ShareButton`), y un menú de opciones desplegable con acciones adicionales

como editar, eliminar o añadir a lista. La visibilidad de los elementos editables está controlada por la variable `isEditingItinerary`, que se determina en función de si el usuario está autenticado y figura como colaborador.

A continuación, se muestra el listado de días y eventos. Esta sección utiliza un acordeón desplegable (`Accordion` de Mantine) para mostrar los días del itinerario, y cada día contiene una lista de eventos, que pueden reordenarse mediante *drag-and-drop* utilizando los componentes `DragDropContext` y `Draggable` de la biblioteca **@hello-pangea/dnd** [48]. Los datos se actualizan en el estado local del componente y se sincronizan con el *back-end* al finalizar el movimiento. Cada evento muestra su título, descripción, horario (si aplica), categoría y una imagen. Cuando el modo de edición está activado, se habilitan botones para añadir, modificar o eliminar eventos, así como para añadir nuevos días o eliminar los existentes. La lógica impide añadir más días que los comprendidos entre las fechas de inicio y fin.

Por último, el componente incluye varios modales de Mantine para tareas específicas: uno para añadir el itinerario a una lista personal; otro para añadir un nuevo evento, con validaciones básicas, selección de categoría y carga de imagen; y otro para editar eventos existentes, precargando los datos del evento seleccionado. En la Figura 49 se muestra, a modo de ejemplo, el modal para añadir un evento.

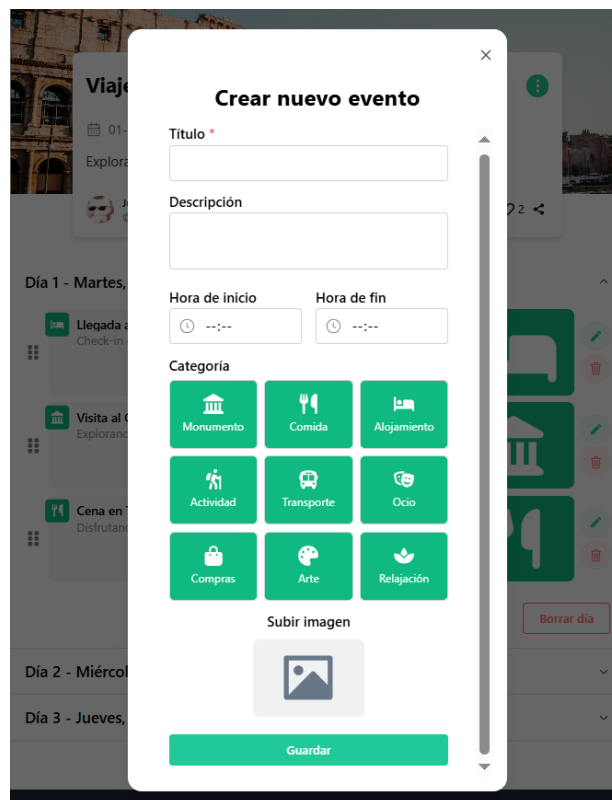


Figura 49. Modal para añadir un evento

La interfaz del componente ha sido diseñada para ser moderna y *responsive*, combinando la biblioteca Mantine con Tailwind CSS. Se han utilizado componentes como `Accordion`, `Menu`, `Textarea`, `DatePickerInput`, `Switch` y `Avatar`, entre otros, para construir una experiencia de usuario fluida tanto en dispositivos móviles como en escritorio. Además,

cuando no se proporciona explícitamente una imagen personalizada, se muestran *placeholders* predeterminados que mantienen la estética visual de la aplicación.

Para gestionar los datos del itinerario, se emplean los *hooks* `useState` y `useEffect` de React. Este último se utiliza, por ejemplo, para realizar la carga inicial de datos al montar el componente (Figura 50), mediante una llamada asíncrona a la función `getById` del servicio `itineraryService`, que solicita a la API la información completa del itinerario a partir de su identificador. El *hook* `useState`, por su parte, se emplea para almacenar el estado del itinerario, los eventos y los días en el componente.

```
useEffect(() => {
  const fetchData = async () => {
    if (userIsLoading) return

    try {
      if (itineraryId) {
        let localIsCollaborator = false
        if (user) {
          try {
            localIsCollaborator = await itineraryService.checkIfCollaborator(itineraryId)
          } catch {
            localIsCollaborator = false
          }
          setIsCollaborator(localIsCollaborator)
        }
        if (!localIsCollaborator) {
          const localItineraryData = await itineraryService.getById(itineraryId)
          setItineraryData(localItineraryData)
          if (localItineraryData.userId) {
            const localUserData = await userService.getById(localItineraryData.userId)
            setUserData(localUserData)
          }
        }
        const localCollaborators = await itineraryService.getCollaborators(itineraryId)
        setCollaborators(localCollaborators)
      }
    } catch (error) {
      if (error instanceof Error) {
        switch (error.message) {
          case 'NotFoundError':
            setNotFoundError(true)
            break
          case 'UnauthorizedError':
            setUnauthorizedError(true)
            break
          default:
            setError('Ocurrió un error inesperado. Por favor, inténtalo de nuevo.')
        }
      } else {
        setError('Ocurrió un error inesperado. Por favor, inténtalo de nuevo.')
      }
    }
  }

  fetchData()
}, [itineraryId, user, userIsLoading])
```

Figura 50. Código para la carga inicial de datos del itinerario

La comunicación en tiempo real entre usuarios colaboradores se gestiona a través de **WebSockets** [10] usando la biblioteca **Socket.IO** [9]. Al abrir la página, si el usuario es colaborador del itinerario, se establece una conexión con el servidor y se une a una sala específica del itinerario mediante el evento `join-itinerary`. A través de distintos eventos como `edit-itinerary`, `add-day`, `delete-day`, `add-event`, `edit-event`, `delete-event` y `reorder-events`, se sincronizan los cambios entre todos los usuarios conectados. Cada vez que se emite o se recibe uno de estos eventos, se actualiza el estado local correspondiente para reflejar los cambios en tiempo real en la interfaz. En la Figura 51 se

puede observar el método `handleAddEvent`, que emite el evento `add-event` mediante WebSockets al resto de colaboradores.

```
const handleAddEvent = async (dayId: string, eventData: Partial<Event>) => {
  if (!itineraryData || !socketRef.current) return

  try {
    socketRef.current.emit('add-event', {
      itineraryId,
      dayId,
      eventData
    })
  } catch {
    console.error('Error adding event')
  }
}
```

Figura 51. Método `handleAddEvent`

4.3.2.5 Página de perfil de un usuario

La página de perfil, mostrada en la Figura 52, permite visualizar la información pública de cualquier usuario y acceder a sus itinerarios, favoritos y listas personalizadas. Esta sección corresponde al componente `Profile`, el cual se integra dentro del `MainLayout` para mantener la coherencia con el resto de páginas principales de la aplicación. La interfaz adapta su contenido en función de si el usuario autenticado está visualizando su propio perfil o el de otro usuario, mostrando botones de edición o seguimiento según corresponda.

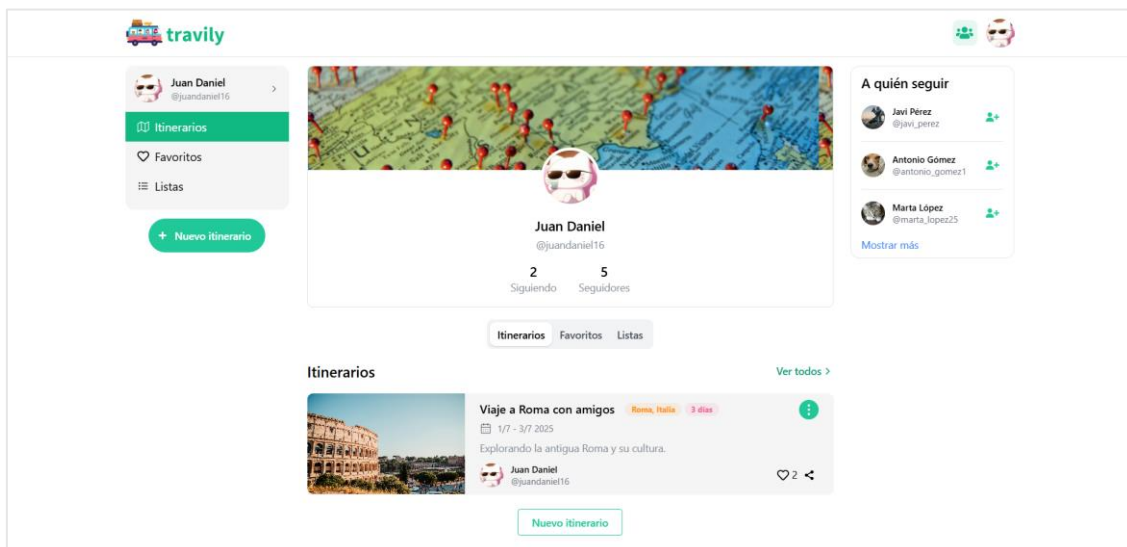


Figura 52. Página de perfil de un usuario

El componente `Profile` gestiona la carga de datos a partir del nombre de usuario (`username`) extraído de la URL, utilizando el método `getByUsername` del `userService`. Una vez obtenido el id del usuario, se recuperan los datos correspondientes a la pestaña activa ("Itinerarios", "Favoritos" o "Listas") mediante llamadas a la API, a través de los servicios `itineraryService` e `itineraryListService`. Esta lógica se implementa con *hooks* de estado como `useState` y `useEffect`.

El contenido de cada pestaña se carga dinámicamente cuando el usuario cambia de vista. En la pestaña "Itinerarios", se muestran los itinerarios creados por el usuario; en "Favoritos", los

itinerarios que ha marcado como favoritos; y en “Listas”, sus colecciones personalizadas. En función del tipo de usuario que accede (visitante o autenticado), se filtra la visibilidad del contenido (público o completo).

En la parte superior se encuentra la cabecera del perfil, que muestra información del usuario: nombre, *username*, avatar, y el número de seguidores y seguidos. También aparece, cuando corresponde, el botón para seguir o dejar de seguir. Bajo esta cabecera se ubica un control de pestañas para navegar entre los diferentes apartados del perfil. La pestaña activa se gestiona mediante un contexto compartido (`useOutletContext`) con el *layout* que envuelve la página de perfil, lo que permite sincronizar el estado de navegación incluso cuando proviene de una redirección con `location.state`. Así, los accesos directos de la columna izquierda en la versión de escritorio redirigen al perfil mostrando la pestaña seleccionada.

Además, el perfil permite visualizar los seguidores y seguidos del usuario mediante un modal, representado en la Figura 53. Esta funcionalidad se activa al navegar a las rutas `/followers` o `/following`, y utiliza los métodos `getFollowers` y `getFollowing` del `userService` para obtener los datos necesarios.

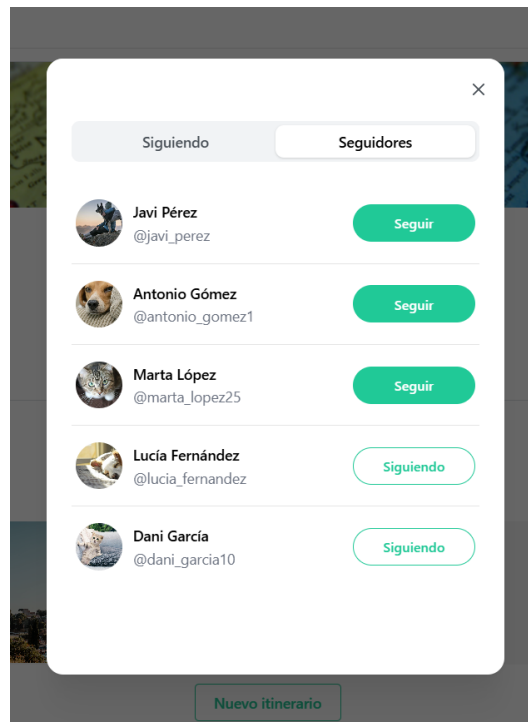


Figura 53. Modal de seguidores de un usuario

El componente también permite eliminar itinerarios o listas desde la página de perfil del usuario. Las funciones `handleDeleteItinerary` y `handleDeleteItineraryList` gestionan estas acciones, eliminando el elemento correspondiente tanto en el servidor como en el estado local, lo que garantiza una experiencia de usuario fluida sin necesidad de recargar toda la página. En caso de error, se muestra un mensaje informativo.

4.3.2.6 Página de búsqueda

La página de búsqueda, mostrada en la Figura 54, permite a los usuarios explorar itinerarios por destinos, así como otros usuarios, mediante un sistema de filtrado y pestañas. Esta

funcionalidad corresponde al componente `SearchResults`, integrado dentro del `MainLayout` para mantener la coherencia con el resto de páginas principales.

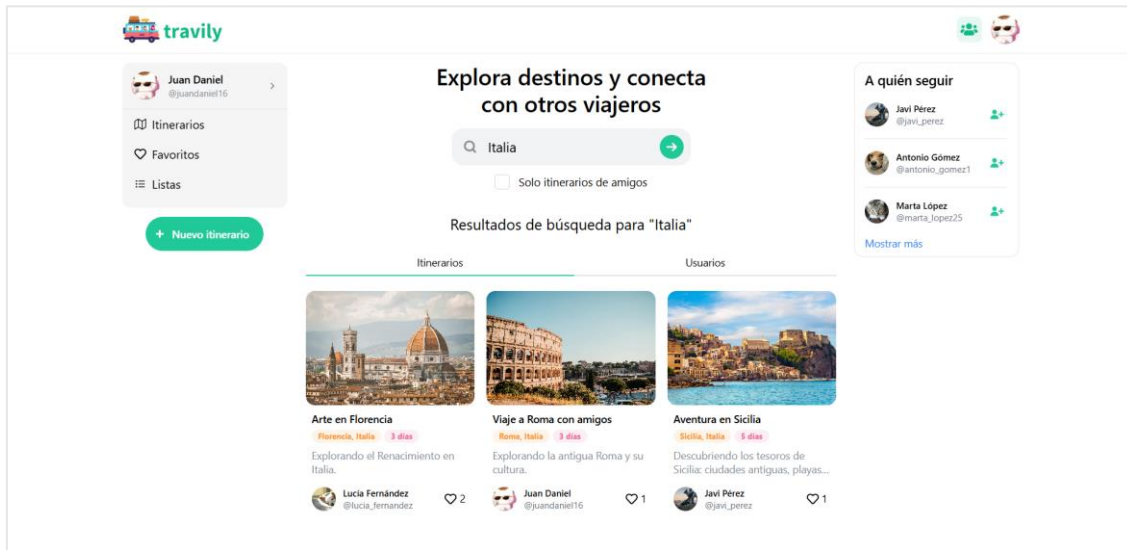


Figura 54. Página de búsqueda

En la parte superior se encuentra el componente `SearchInput`, que permite introducir términos de búsqueda. Para facilitar la localización de destinos, se muestran sugerencias geográficas obtenidas mediante la API pública de **GeoNames** [34]. Si el usuario está autenticado, se habilita una opción adicional: un *checkbox* para limitar los resultados únicamente a itinerarios creados por amigos. Este filtro se gestiona mediante el estado `onlyFriends` y se sincroniza con los parámetros de la URL.

Los resultados de búsqueda se organizan en pestañas (`Tabs`) para alternar entre itinerarios y usuarios. La pestaña activa se sincroniza con los parámetros de búsqueda (`q`, `type` y `onlyFriends`) mediante el *hook* `useSearchParams`, lo que permite una navegación coherente y la posibilidad de redirigir directamente con filtros predefinidos.

La obtención de resultados se gestiona mediante un `useEffect` que se activa al cambiar la consulta (`query`), el tipo de búsqueda (`type`) o el filtro `onlyFriends`. En función del tipo seleccionado (*itinerary* o *user*), se realizan llamadas a los servicios correspondientes: `itineraryService.getAll`, que recupera itinerarios filtrados por localización (basada en `query`) y ordenados por popularidad, incluyendo el parámetro `followedBy` con el ID del usuario autenticado si el filtro `onlyFriends` está activo; o bien `userService.getAll`, que busca usuarios cuyo nombre o nombre de usuario coincida con el término de búsqueda.

Durante la carga de resultados se muestra un *spinner* (`Loader`), y se manejan estados de error o ausencia de resultados con mensajes informativos. Los itinerarios se muestran en un *grid* mediante el componente `ItineraryCard`, mientras que los usuarios se listan con `UsersList`, que además permite seguir o dejar de seguir directamente desde los resultados, utilizando la función `handleFollow`. El diseño es *responsive*, adaptándose correctamente a dispositivos móviles.

4.4 Pruebas

Durante el desarrollo de la aplicación, se han llevado a cabo pruebas de cada nueva funcionalidad implementada, verificando que todas las partes del sistema respondieran correctamente ante distintos escenarios de uso.

Para validar el *back-end*, se ha utilizado la aplicación **Postman** [26] como herramienta principal, permitiendo enviar peticiones HTTP a la API y analizar la respuesta de los diferentes *endpoints* de manera controlada. Las peticiones se organizaron en colecciones agrupadas por controlador, lo que facilitó su revisión y mantenimiento a lo largo del proyecto. La Figura 55 muestra la interfaz de aplicación, con las colecciones creadas y sus respectivas peticiones.

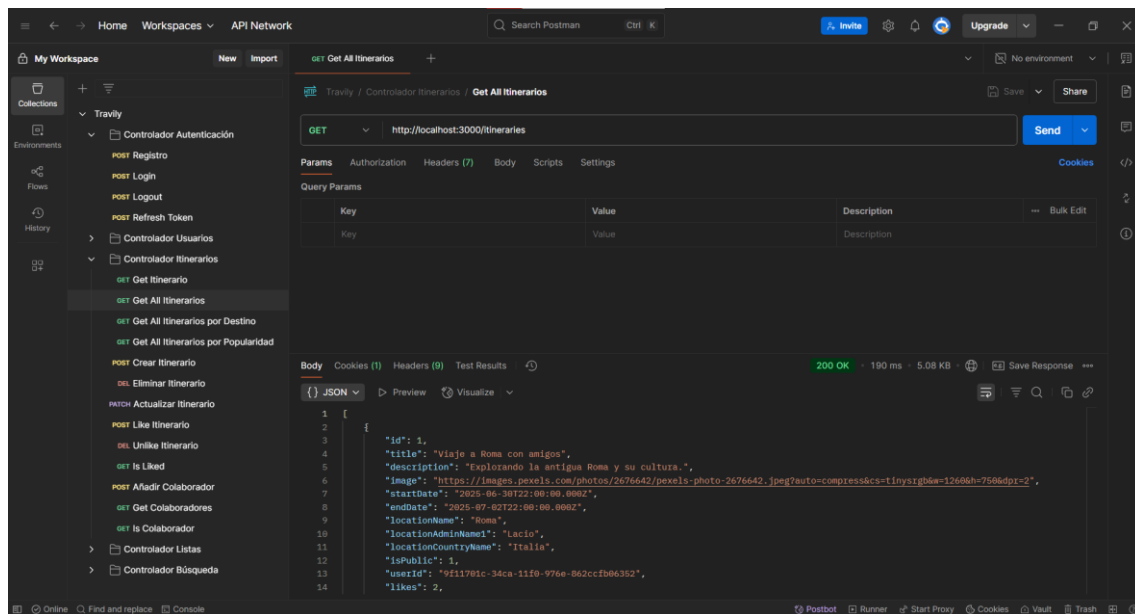


Figura 55. Aplicación Postman

En cuanto al *front-end*, se realizaron comprobaciones manuales utilizando las herramientas de desarrollo del navegador. Esto permitió simular distintos tamaños de pantalla y dispositivos, asegurando que la interfaz de la aplicación mantuviera su diseño responsive y que la experiencia de usuario fuera consistente. Además, se interactuó con la aplicación desde distintos tipos de usuario: como creador de un itinerario, como colaborador, como usuario registrado sin permisos de edición y como usuario no registrado, con el objetivo de verificar que la lógica de permisos y edición funcionara correctamente en cada caso.

Este enfoque ha permitido asegurar la estabilidad y coherencia del sistema durante todo el proceso de desarrollo, sin necesidad de implementar pruebas unitarias u otras pruebas automatizadas más complejas, priorizando la eficiencia sin comprometer la calidad del producto.

4.5 Despliegue

Para el despliegue de la aplicación se han utilizado distintos servicios en la nube, uno para cada componente principal: *front-end*, *back-end* y almacenamiento de datos. Esta separación permite una gestión más flexible, además de asegurar disponibilidad y escalabilidad.

En primer lugar, el *front-end* se ha desplegado en **Vercel** [49], una plataforma que permite compilar y publicar aplicaciones web directamente desde un repositorio Git. Gracias a su integración con GitHub y a su sistema automático de *builds*, cada vez que se realiza un *push*, la aplicación se despliega automáticamente. Además, se han configurado las variables de entorno necesarias para la conexión con el *back-end*.

El *back-end* se ha desplegado en **Render** [50], un servicio que también permite desplegar aplicaciones desde un repositorio Git. Detecta automáticamente el entorno de ejecución Node.js, instala las dependencias necesarias y ejecuta el servidor. Asimismo, permite definir variables de entorno para la conexión con la base de datos y para la gestión de tokens JWT en el sistema de autenticación.

En cuanto al almacenamiento de datos, el sistema combina una base de datos relacional y un servicio en la nube para archivos multimedia. Por un lado, la base de datos MySQL se ha desplegado en **Aiven** [51], una plataforma de bases de datos gestionadas en la nube. Desde su panel de administración se ha creado una instancia de MySQL, configurando los parámetros de seguridad y acceso necesarios para la conexión desde el *back-end*. Por otro lado, las imágenes subidas por los usuarios se almacenan en **Cloudinary** [39], un servicio especializado en la gestión de archivos multimedia. Desde el servidor, las imágenes se procesan y se envían directamente a Cloudinary, donde se almacenan en carpetas organizadas según el tipo de recurso (avatares, itinerarios, eventos y listas).

Esta configuración permite que todos los servicios estén disponibles en entornos separados pero conectados entre sí mediante URLs públicas y credenciales seguras. La comunicación entre el cliente y el servidor se realiza mediante peticiones HTTP, permitidas gracias a la configuración de CORS en el *back-end*.

La aplicación Travily está disponible públicamente en la siguiente URL: <https://travily.net>

5 Conclusiones y vías futuras

5.1 Conclusiones

Una vez finalizado este proyecto, se puede afirmar que se han cumplido con éxito todos los objetivos marcados en la fase de análisis. Se ha desarrollado una aplicación web *responsive* que permite crear y compartir itinerarios de viaje con otros usuarios. Durante el proceso, se ha realizado un estudio de la competencia, se han determinado las tecnologías a utilizar y se ha definido el alcance del sistema especificando sus requisitos mediante casos de uso. Además, se han definido tanto el diseño como la arquitectura del sistema, y se han creado mockups para ilustrar el aspecto final de la aplicación. Posteriormente, se llevó a cabo la implementación de la aplicación, seguida de las pruebas correspondientes y, finalmente, su despliegue. Todo el proceso ha sido documentado en esta memoria.

La aplicación resultante se presenta como una red social que permite a los usuarios crear, compartir y gestionar itinerarios de viaje de forma colaborativa. La edición en tiempo real fomenta la interacción y cooperación entre usuarios en el proceso de planificación. Además, la plataforma proporciona una interfaz intuitiva y accesible, lo que facilita su uso tanto para viajeros frecuentes como para aquellos que recién comienzan a organizar sus viajes.

Sin embargo, aunque el sistema actual es completamente funcional y útil para la gestión de itinerarios, existen áreas de mejora que podrían potenciar aún más la plataforma. Características adicionales, como la integración con otros servicios de pago o la incorporación de funcionalidades avanzadas, podrían convertir esta herramienta en una solución aún más robusta y competitiva en el sector de la planificación de viajes.

Por último, cabe destacar el uso de herramientas basadas en inteligencia artificial generativa, como ChatGPT, a lo largo del desarrollo del proyecto. Estas han contribuido a agilizar el trabajo y mejorar la calidad, especialmente en el desarrollo y revisión de fragmentos de código, la comprensión de errores y la búsqueda de soluciones más eficientes. No obstante, su uso ha requerido un juicio crítico constante, ya que no siempre las respuestas han sido precisas o adecuadas al contexto, por lo que ha sido necesario contrastarlas con fuentes fiables y aplicar criterios propios. Asimismo, se han utilizado para mejorar la redacción y corrección de textos en este documento. En conjunto, este proyecto ha permitido adquirir una mayor destreza en el uso de estas soluciones, cada vez más presentes en el ámbito profesional y académico.

5.2 Vías futuras

Como se mencionó previamente, la plataforma desarrollada ofrece una base sólida para la creación y gestión de itinerarios de viaje. Sin embargo, durante el desarrollo del proyecto se plantearon diversas funcionalidades que, debido a limitaciones de tiempo y recursos, no pudieron ser implementadas.

Una de las características más relevantes que se podría integrar es un chat privado entre los colaboradores de un itinerario. Este chat permitiría a los usuarios comunicarse directamente dentro de la aplicación para discutir cualquier ajuste o cambio que deseen realizar en el itinerario, sin necesidad de recurrir a medios externos. Esta funcionalidad facilitaría la

coordinación y la toma de decisiones conjuntas, especialmente en itinerarios complejos con múltiples participantes.

Asimismo, la incorporación de un sistema de notificaciones sería fundamental para mejorar la interacción entre los usuarios. Este sistema podría notificar a los viajeros sobre diversas acciones, como la creación, edición o eliminación de itinerarios, cambios en las fechas de eventos programados, nuevos seguidores, los 'me gusta' recibidos en itinerarios o listas, y las invitaciones para colaborar en itinerarios. De este modo, los usuarios se mantendrían informados y activos dentro de la plataforma.

Otro aspecto clave para enriquecer la plataforma sería la internacionalización de la aplicación. La posibilidad de cambiar el idioma de la interfaz facilitaría el acceso a un público más amplio, permitiendo que usuarios de distintas partes del mundo puedan disfrutar de la herramienta.

En cuanto a la integración con servicios externos, la conexión con la API de pago de Google Maps podría resultar de gran utilidad, ya que permitiría a los usuarios obtener información actualizada sobre opciones de transporte, alojamiento y precios directamente desde la plataforma.

Por último, otra mejora importante sería la opción de dejar comentarios en itinerarios y eventos. De este modo, los usuarios podrían compartir sus experiencias y recomendaciones sobre lugares y actividades, lo que fomentaría una mayor interacción y ayudaría a otros viajeros a tomar decisiones.

6 Bibliografía

- [1] «Se dispara la búsqueda online de viajes en España: crece un 20% la audiencia diaria de estos sites», NIQ. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://nielseniq.com/global/es/news-center/2024/se-dispara-la-busqueda-online-de-viajes-en-espana-crece-un-20-la-audiencia-diaria-de-estos-sites/>
- [2] «Triplt - Highest-rated trip planner and flight tracker». Accedido: 11 de diciembre de 2024. [En línea]. Disponible en: <https://www.tripit.com/web>
- [3] «Wanderlog travel planner: free vacation planner and itinerary app». Accedido: 27 de diciembre de 2024. [En línea]. Disponible en: <https://wanderlog.com/>
- [4] «Stippl: The All In One Adventure Planner». Accedido: 27 de diciembre de 2024. [En línea]. Disponible en: <https://stippl.io/>
- [5] «JavaScript | MDN». Accedido: 18 de marzo de 2025. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/JavaScript>
- [6] «Node.js — Ejecuta JavaScript en cualquier parte». Accedido: 18 de marzo de 2025. [En línea]. Disponible en: <https://nodejs.org/es>
- [7] «¿Qué es una API de RESTful? - Explicación de API de RESTful - AWS», Amazon Web Services, Inc. Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://aws.amazon.com/es/what-is/restful-api/>
- [8] «Express - Node.js web application framework». Accedido: 18 de marzo de 2025. [En línea]. Disponible en: <https://expressjs.com/>
- [9] «Socket.IO». Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://socket.io/>
- [10] «The WebSocket API (WebSockets) - Web APIs | MDN». Accedido: 26 de marzo de 2025. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [11] «TypeScript-first schema validation with static type inference», GitHub. Accedido: 24 de abril de 2025. [En línea]. Disponible en: <https://github.com/colinhacks/zod>
- [12] «npm | Home». Accedido: 18 de marzo de 2025. [En línea]. Disponible en: <https://www.npmjs.com/>
- [13] «MySQL». Accedido: 18 de marzo de 2025. [En línea]. Disponible en: <https://www.mysql.com/>
- [14] «MySQL en detalle: guía para dominar esta potente base de datos». Accedido: 18 de marzo de 2025. [En línea]. Disponible en: <https://www.oracle.com/es/mysql/what-is-mysql/>
- [15] «React». Accedido: 19 de marzo de 2025. [En línea]. Disponible en: <https://es.react.dev/>
- [16] «JavaScript With Syntax For Types.» Accedido: 19 de marzo de 2025. [En línea]. Disponible en: <https://www.typescriptlang.org/>
- [17] «Technology | 2024 Stack Overflow Developer Survey». Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://survey.stackoverflow.co/2024/technology/>
- [18] «Virtual DOM and Internals – React». Accedido: 19 de marzo de 2025. [En línea]. Disponible en: <https://legacy.reactjs.org/docs/faq-internals.html>
- [19] «HTML: Lenguaje de etiquetas de hipertexto | MDN». Accedido: 19 de marzo de 2025. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/HTML>
- [20] «Tailwind CSS - Rapidly build modern websites without ever leaving your HTML.» Accedido: 19 de marzo de 2025. [En línea]. Disponible en: <https://tailwindcss.com/>
- [21] «CSS | MDN». Accedido: 19 de marzo de 2025. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/CSS>
- [22] «Mantine». Accedido: 19 de marzo de 2025. [En línea]. Disponible en: <https://mantine.dev/>
- [23] «Visual Studio Code - Code Editing. Redefined». Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://code.visualstudio.com/>

- [24] «Figma: La herramienta de diseño de interfaz colaborativa», Figma. Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://www.figma.com/es-es/>
- [25] «MySQL :: MySQL Workbench». Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://www.mysql.com/products/workbench/>
- [26] «Postman API Platform». Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://www.postman.com/>
- [27] «Git». Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://git-scm.com/>
- [28] «GitHub · Build and ship software on a single, collaborative platform», GitHub. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://github.com/>
- [29] «ChatGPT». Accedido: 26 de marzo de 2025. [En línea]. Disponible en: <https://chatgpt.com>
- [30] «MVC - Glosario de MDN Web Docs: Definiciones de términos relacionados con la Web | MDN». Accedido: 24 de marzo de 2025. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Glossary/MVC>
- [31] «dotenv», npm. Accedido: 25 de abril de 2025. [En línea]. Disponible en: <https://www.npmjs.com/package/dotenv>
- [32] «mysql2», npm. Accedido: 24 de abril de 2025. [En línea]. Disponible en: <https://www.npmjs.com/package/mysql2>
- [33] «What is an ORM (Object Relational Mapper)?», Prisma's Data Guide. Accedido: 24 de abril de 2025. [En línea]. Disponible en: <https://www.prisma.io/dataguide/types/relational/what-is-an-orm>
- [34] «GeoNames». Accedido: 29 de mayo de 2025. [En línea]. Disponible en: <https://www.geonames.org/>
- [35] Auth0, «JSON Web Tokens - jwt.io», JSON Web Tokens - jwt.io. Accedido: 25 de abril de 2025. [En línea]. Disponible en: <https://jwt.io>
- [36] «jsonwebtoken», npm. Accedido: 25 de abril de 2025. [En línea]. Disponible en: <https://www.npmjs.com/package/jsonwebtoken>
- [37] «bcrypt», npm. Accedido: 25 de abril de 2025. [En línea]. Disponible en: <https://www.npmjs.com/package/bcrypt>
- [38] «Cross-site scripting (XSS) - Security on the web | MDN». Accedido: 16 de mayo de 2025. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>
- [39] «Image and Video Upload, Storage, Optimization and CDN», Cloudinary. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://cloudinary.com/>
- [40] «multer-storage-cloudinary», npm. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://www.npmjs.com/package/multer-storage-cloudinary>
- [41] «multer», npm. Accedido: 25 de abril de 2025. [En línea]. Disponible en: <https://www.npmjs.com/package/multer>
- [42] «Uso de Fetch - Referencia de la API Web | MDN». Accedido: 27 de abril de 2025. [En línea]. Disponible en: https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- [43] «React Router Official Documentation». Accedido: 27 de abril de 2025. [En línea]. Disponible en: <https://reactrouter.com/>
- [44] «Vite», vitejs. Accedido: 27 de abril de 2025. [En línea]. Disponible en: <https://vite.dev>
- [45] «Escribir marcado con JSX – React». Accedido: 27 de abril de 2025. [En línea]. Disponible en: <https://es.react.dev/learn/writing-markup-with-jsx>
- [46] «useState – React». Accedido: 29 de abril de 2025. [En línea]. Disponible en: <https://react.dev/reference/react/useState>
- [47] «useNavigate». Accedido: 29 de abril de 2025. [En línea]. Disponible en: <https://reactrouter.com/api/hooks/useNavigate>
- [48] «@hello-pangea/dnd», npm. Accedido: 29 de abril de 2025. [En línea]. Disponible en: <https://www.npmjs.com/package/@hello-pangea/dnd>

- [49] «Vercel: Build and deploy the best web experiences with the Frontend Cloud», Vercel. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://vercel.com/home>
- [50] «Cloud Application Platform», Render. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://render.com>
- [51] «Aiven - Your AI-ready Open Source Data Platform», Aiven. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: <https://aiven.io>